Open Engineering Architecture & Agentic Systems Series · Faculty & Instructor Edition

From Vibe Coding to Agentic Engineering

Lecturer & Professor Instructor Manual: Curriculum Design, Slide-by-Slide Delivery Playbook, Live Projector Demos, Lab Answer Keys & Assessment Rubrics (agy & claude)

Author: Alejandro Kaspar — AI Forward Deployed Engineer · Licensed under CC BY 4.0 & Apache 2.0

How to Use This Instructor Manual Across University Seminars & Engineering Bootcamps

This manual is the companion faculty and instructor guide for the two-phase technical course *From Vibe Coding to Agentic Engineering*. Whereas the student coursebook and 42-slide lecture deck present the material from the learner's perspective, this volume is written directly from one instructor to another. It provides turnkey pacing schedules, minute-by-minute lecture narratives, eight live projector terminal demonstrations, visual walkthrough guides for every empirical chart on the slides, Socratic discussion prompts, complete answer keys and teaching-assistant (TA) debugging playbooks for all eight Python labs, open-ended exam rubrics, and an annotated literature briefing:

- **Section 1 — Course Architecture, Pedagogical Arc, Pacing Syllabi & Classroom Provisioning:** Explains why students must experience Phase I daily steering before Phase II harness architecture, provides three turnkey syllabi (8-Week University Seminar, 2-Day Intensive Bootcamp, and 4-Session Staff Workshop), and details offline autograding via `./self_diagnose_all.sh` across the `main`, `agy`, and `claude` tracks.
- **Section 2 — Phase I Lecture Delivery Playbook (Modules 1–4, Slides 01–19):** Minute-by-minute teaching notes, Projector Demos 1–4, line-by-line guides for walking students through the visual evolution timeline (Slide 03), the six eras (Slide 04), the three model-release-pace charts (Slide 05), and the Context Window Physics curve (Slide 16), plus Socratic prompts to diagnose common student misconceptions.
- **Section 3 — Phase II Lecture Delivery Playbook (Modules 5–8, Slides 20–42):** Instructor scripts and Projector Demos 5–8 covering the 60-line context map, progressive-disclosure skills, Meta-MCP Code Mode, context-firewall subagents, Jujutsu (`jj`) conflict algebra, and the multi-slide empirical dashboards for `SkillsBench`, `Omnigent`, `autoresearch`, and Recursive Self-Improvement.
- **Section 4 — Complete Lab Facilitation Guide, Answer Key & TA Debugging Playbook (Labs 01–08):** Detailed walkthrough of every intentional flaw in `starter/`, the reference architecture in `expected_output/`, the 48 adversarial requirements (`REQ-0101`–`REQ-0806`), and Socratic TA coaching hints for unblocking students in lab workshops.
- **Section 5 — Capstone Specification, Open-Ended Design Exam Bank & Annotated Literature Guide:** A 100-point capstone project rubric, eight open-ended architectural design exam questions with model grading keys, and the instructor's annotated primary-source literature briefing.

Contents

1 Course Architecture, Pedagogical Philosophy, Pacing Schedules & Classroom Setup **3**

1.1 The Pedagogical Arc: Why Phase I Daily Steering Must Precede Phase II Harness Design . . . 3

1.2 Three Turnkey Pacing Syllabi: University Seminar, 2-Day Bootcamp & Staff Workshop . . . 4

1.3 Dual-Track Classroom Provisioning (agy & claude) & Deterministic Offline Autograding . . . 5

2 Phase I Lecture Delivery Playbook (Modules 1–4, Slides 01–19): Live Demos, Chart Walkthroughs & Socratic Prompts **7**

2.1 Module 1 (Slides 01–07): Language, Code as a Conceptual Model, The Visual Evolution Timeline & The 17-Day Model Pace . . . 7

2.2 Module 2 (Slides 08–11): Inside the Agentic Loop, agy & claude 101, and Wire-Trace Inspection . . . 9

2.3 Module 3 (Slides 12–15): Directing the Daily Workflow (RePPIT), Orthogonal Proposals & Context Reset . . . 9

2.4 Module 4 (Slides 16–19): Context Window Physics, The 40% "Dumb Zone" & Frequent Intentional Compaction 10

3 Phase II Lecture Delivery Playbook (Modules 5–8, Slides 20–42): Live Demos, Chart Walkthroughs & Socratic Prompts 13

3.1 Module 5 (Slides 20–23): The 2×2 Cybernetic Control Matrix, The 60-Line Map Rule & Cross-Session Auto-Memory 13

3.2 Module 6 (Slides 24–29): Progressive-Disclosure Skills, `chub` Grounding, Meta-MCP Code Mode & `SkillsBench` 14

3.3 Module 7 (Slides 30–34): Context-Firewall Subagents, PR Triage, `llm-council` & Jujutsu (`jj`) VCS 15

3.4 Module 8 (Slides 35–42): 4-Tier Governance, `Omnigent`, Remediation Linters, `Autoresearch` & RSI Charts . . . 15

4 Complete Lab Facilitation Guide, Instructor Answer Key & TA Debugging Playbook (Labs 01–08) 18

4.1 How to Facilitate the 90-Minute Hands-On Lab Workshops 18

4.2 Phase I Lab Answer Keys, Everyday Metaphors & 3-Rung TA Hint Ladders (Labs 01–04) 18

4.3 Phase II Lab Answer Keys, Everyday Metaphors & 3-Rung TA Hint Ladders (Labs 05–08) 21

5 Capstone Project Specification, Grading Rubrics, Open-Ended Design Exam Bank & Annotated Literature Guide 25

5.1 Capstone Project Specification ("Design & Verify a Production Agent Harness") & 100-Point Rubric 25

5.2 Open-Ended Conceptual & Architectural Design Exam Bank with Model Grading Keys 25

5.3 Instructor's Annotated Literature & Primary Sources Guide 28

1 Course Architecture, Pedagogical Philosophy, Pacing Schedules & Classroom Setup

1.1 The Pedagogical Arc: Why Phase I Daily Steering Must Precede Phase II Harness Design

Teaching the Human-in-the-Loop Before Architecting the Loop. When introducing agentic software engineering to senior undergraduates, graduate seminar students, or practicing industry engineers, the most common curriculum mistake is starting directly with multi-agent orchestration, custom Model Context Protocol (MCP) servers, or autonomous overnight loops. Students who have never inspected a raw JSONL wire trace or experienced attention degradation inside a bloated chat session treat harnesses as abstract bureaucracy. This course is deliberately structured as a two-phase progression so learners first experience the physical limits of interactive coding agents firsthand before building the engineering systems that govern them.

Phase I (Modules 1–4): Mastering Interactive Steering and Context Physics. In the first half of the course, students work directly inside a single coding session using Google Antigravity (`agy`) or Anthropic Claude Code (`claude`). You will guide them through four foundational shifts in how they think about code and language:

1. **Module 1 (Language, Code & Earned Autonomy):** Students discover why conversational prompts can coordinate deterministic software changes at all, why passive "vibe reviewing" accumulates severe cognitive debt, and how software engineering fundamentals (latency bounds, consistency invariants, idempotency, and integer minor units) separate high-agency conductors from passive prompt typists.
2. **Module 2 (Inside the Agentic Loop & Wire Traces):** Before writing custom tools, students open the hood of the three-layer agent stack. By inspecting an actual HTTP/JSONL wire trace, they see that mounting 150 granular CRUD tools injects over 10,000 lines of schema definitions before a single line of user code is read, motivating the shift to composable "Swiss Army Knife" primitives (`Bash`, `Read`, `Edit`, `Grep`, and `LSP`).
3. **Module 3 (The 5-Step `RePPIT` Daily Workflow):** Once the agent loop is transparent, students learn to calibrate ceremony to task complexity. For multi-file features, they practice the five-step `Research -> Propose -> Plan -> Implement -> Verify` (`RePPIT`) workflow, including Socratic requirement interviewing (`/grill-me` or Plan Mode), generating two orthogonal proposals, purging rejected proposals from context, and locking an explicit `Out of Scope / Do NOT Touch` guardrail list.
4. **Module 4 (Context Window Physics & The 40% "Dumb Zone"):** Phase I culminates in the physical law that governs every subsequent design decision: NextAction = LLM(ContextWindow). Students measure how unfiltered test logs and exploration dead-ends push utilization past the 40% Smart Zone ceiling into Context Rot, and they practice shell output redirection (`> run.log 2>&1`) and Frequent Intentional Compaction (`RPI`) into line-cited `research.md` and `plan.md` artifacts.

Phase II (Modules 5–8): Transitioning to Cybernetic Harness Engineering. Once students feel the friction of manually managing context budgets and repeating project instructions across fresh sessions, Phase II introduces the formal discipline of **Harness Engineering** (Agent = Model + Harness). Every module in Phase II automates and hardens a pain point discovered during Phase I (see Figure 1 below):

- **Module 5 (Layer 1 — Standing Context Maps & Auto-Memory):** Replaces repetitive session setup with a $\leq$ 60-line root `CLAUDE.md` or `GEMINI.md` map (staying well below the 150-instruction compliance cliff), glob-scoped rules, and provenance-tagged cross-session memory (`MEMORY.md` and `KNOWLEDGE.md`).
- **Module 6 (Layer 2 — Progressive-Disclosure Skills, `chub` & Meta-MCP Code Mode):** Replaces monolithic prompt dumps and 150-tool schema bloat with three-level `agentskills.io` packages, curated

self-annotating API documentation (`chub get` and `chub annotate`), and Meta-MCP `search` + `execute` Code Mode (> 90% token savings).

- **Module 7 (Layer 3 — Subagents as Context Firewalls, Peer Councils & Agentic VCS):** Replaces single-window search pollution and single-model review bias with isolated read-only subagents (50,000 → 500 token compression), five-lens pull-request triage (confidence ≥ 80), three-stage anonymized `llm-council` Borda ranking, and parallel workspace isolation via `git worktree` and colocated Jujutsu (`jj`).
- **Module 8 (Layer 4 — 4-Tier Governance, Remediation Linters, `Autoresearch` & Program Design):** Synthesizes the entire stack into governed autonomous loops: `PreToolUse` Exit-Code-2 hard floors, durable execution replay (`Temporal`, `DBOS`, `Restate`) and cross-harness meta-harnesses (`Omnigent`), OpenAI-style `[VIOLATION] -> [WHY] -> [HOW TO FIX]` structural linters, Karpathy's `autoresearch` keep-or-revert ratchet, and four-phase front-loaded Program Design to prevent dark-factory collapse.

Figure 1.1 Blueprint: Two-Phase Pedagogical Progression & 8-Module Dependency Map

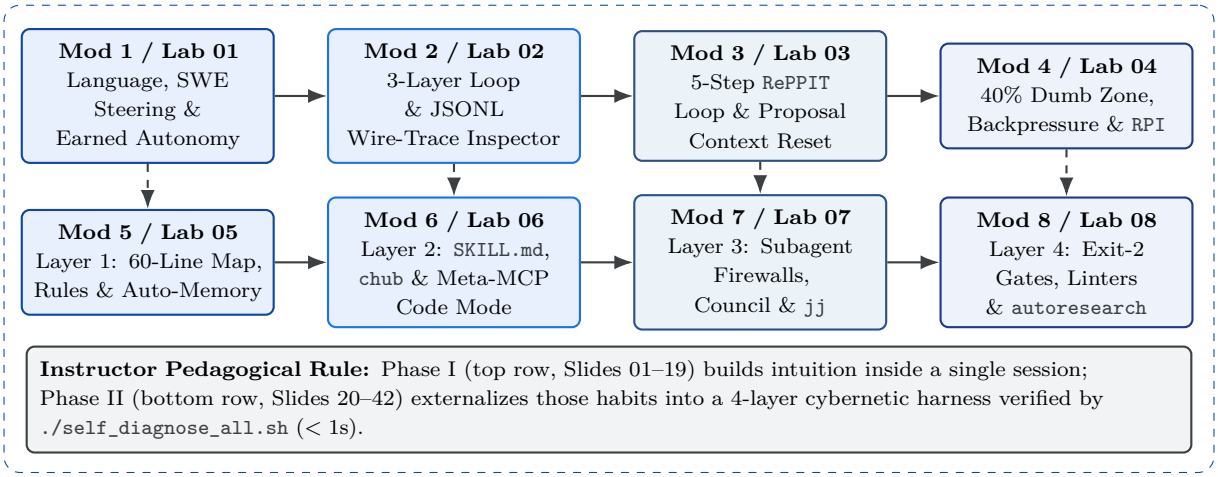


Figure 1: The complete Two-Phase Pedagogical Progression showing how Phase I daily workflow foundations (Modules 1–4, Labs 01–04) directly motivate each Phase II Harness Engineering layer (Modules 5–8, Labs 05–08).

1.2 Three Turnkey Pacing Syllabi: University Seminar, 2-Day Bootcamp & Staff Workshop

Adapting the Curriculum to Academic and Industrial Formats. The 42-slide deck, 8 modular lecture videos, and 8 self-diagnosing Python labs are designed to scale cleanly across three standard teaching formats (Table 1):

1. **Format 1 — 8-Week University Seminar (16 Sessions, 3 Credits):** Ideal for senior undergraduate or graduate computer science courses (modeled after modern software developer seminars). Each week pairs one 75-minute interactive lecture and live projector demonstration (Session A) with one 90-minute supervised coding lab workshop (Session B), culminating in a multi-week Capstone Harness Project graded in Week 8.
2. **Format 2 — 2-Day Intensive Practitioner Bootcamp (16 Hours Total):** Designed for industry software engineers, forward-deployed engineers, and platform teams. Day 1 covers Phase I (Modules 1–4 and Labs 01–04) in four 2-hour blocks (45 minutes of lecture + 15 minutes of live projector demo + 60 minutes of paired lab implementation). Day 2 covers Phase II (Modules 5–8 and Labs 05–08), concluding with a full-cohort run of `./self_diagnose_all.sh`.

3. **Format 3 — 4-Session Staff & Tech-Lead Architecture Workshop (Four 2-Hour Sessions):** Tailored for staff engineers, engineering managers, and AI platform leads who are standardizing repository harnesses across an organization. Each 2-hour session pairs two adjacent modules (Session 1: Modules 1–2; Session 2: Modules 3–4; Session 3: Modules 5–6; Session 4: Modules 7–8), focusing lab time on auditing the team's own production repositories against the course rubrics.

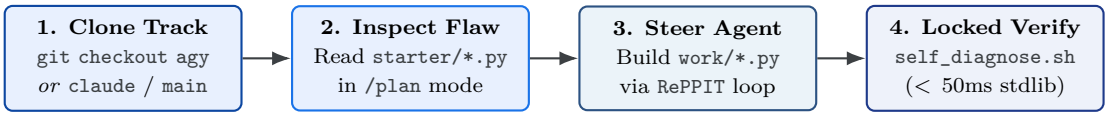
1.3 Dual-Track Classroom Provisioning (agy & claude) & Deterministic Offline Autograding

Why Dual-Branch Parity (agy and claude) Eliminates Classroom Lock-In. In any university classroom or corporate engineering cohort, students arrive with heterogeneous toolchains: some teams standardize on Google Antigravity (`agy` CLI, Mission Control Agent Manager, and `GEMINI.md`), while others use Anthropic Claude Code (`claude` CLI and `CLAUDE.md`). Rather than forcing instructors to maintain two divergent repositories, the course repository provides 1-to-1 structural parity across three branches:

- `main` **Branch (Dual-Harness Reference):** Contains both `GEMINI.md` and `CLAUDE.md` (with `AGENTS.md` symlinked), side-by-side configuration trees (`.agents/` and `.claude/`), and dual-track walkthrough instructions in every lab's `WALKTHROUGH.md`.
- `agy` **Branch (Google Antigravity Native Track):** Pre-configured for learners running `agy --workspace .`, highlighting `.agents/rules/`, `.agents/skills/`, `.agents/agents/`, `.agents/workflows/feature-dev.md`, and `~/.gemini/antigravity-cli/settings.json`.
- `claude` **Branch (Anthropic Claude Code Native Track):** Pre-configured for learners running `claude --settings .claude/learner.settings.json`, highlighting `.claude/rules/`, `.claude/skills/`, `.claude/agents/`, and `.claude-plugin/plugin.json`.

Zero-API-Key Deterministic Autograding in Under One Second. A frequent operational headache when teaching AI engineering courses is lab flakiness caused by live API rate limits (`HTTP 429`), campus Wi-Fi outages, or variable student token budgets. Every lab in this course separates **interactive agent steering** from **deterministic harness verification**. Students use their coding agent (`agy` or `claude`) to research, plan, and implement each lab's Python harness module inside `work/`, but the grading harness itself runs 100% offline using only the Python 3.10+ standard library (`unittest`).

Instructor Provisioning Checklist & Sub-Second Classroom Autograder




```
# Verify the entire instructor baseline (8 labs, 72 unit tests, <1.0s):
$ ./self_diagnose_all.sh
# [ALL PASSED] 8/8 Hands-On Vibe-Coding-Course Labs Verified!

# Grade a student's work/ directory in Lab 01 (locks adversarial_tests/):
$ ./labs/lab_01_vibe_coding_and_earned_autonomy/self_diagnose.sh work
```

Preventing Agent Reward Hacking During Student Labs. Notice how `self_diagnose.sh` executes a critical pedagogical defense before running `unittest`: it temporarily locks `adversarial_tests/` as read-only (`chmod -R a-w adversarial_tests`) and verifies that the student's agent did not modify the test suite to make failing assertions pass. Point this mechanism out during Week 1 orientation—it foreshadows the immutable evaluation sensor (`prepare.py`) and Exit-Code-2 hard floors that students will architect themselves in Module 8.

Unit & Slides	8-Week University Seminar (16 × 80m)	2-Day Practitioner Bootcamp (2 × 8h)	4-Session Staff Workshop (4 × 2h)	Assigned Lab & Primary Deliverable
Module 1 (Slides 01–07)	**Week 1:** Lec 1A (Language, Fowler, 17d Pace) + Lab 1B Workshop.	**Day 1, 09:00–11:00:** Phase I Kickoff, Demo 1 & Lab 01.	**Session 1 (Part A):** Pillar 2 Steering & Earned Autonomy.	**Lab 01:** `prompt_steerer.py` (`REQ-0101..0106`).
Module 2 (Slides 08–11)	**Week 2:** Lec 2A (`agy/claude` 101 & Wire Trace) + Lab 2B.	**Day 1, 11:00–13:00:** Wire Trace Inspection, Demo 2 & Lab 02.	**Session 1 (Part B):** Swiss-Army Primitives vs. Schema Bloat.	**Lab 02:** `wire_trace_inspector.py` (`REQ-0201..0206`).
Module 3 (Slides 12–15)	**Week 3:** Lec 3A (5-Step `RePPIT` & Reset) + Lab 3B Workshop.	**Day 1, 14:00–16:00:** `/grill-me`, Plan Mode, Demo 3 & Lab 03.	**Session 2 (Part A):** `RePPIT` Governance & Model Routing.	**Lab 03:** `reppit_orchestrator.py` (`REQ-0301..0306`).
Module 4 (Slides 16–19)	**Week 4:** Lec 4A (40% Dumb Zone & `RPI`) + Lab 4B + Midterm Check.	**Day 1, 16:00–18:00:** Context Physics, Demo 4 & Lab 04.	**Session 2 (Part B):** Backpressure & Leverage Pyramid.	**Lab 04:** `context_compactor.py` (`REQ-0401..0406`).
Module 5 (Slides 20–23)	**Week 5:** Lec 5A (2×2 Matrix, 60-Line Map & Memory) + Lab 5B.	**Day 2, 09:00–11:00:** Phase II Kickoff, Demo 5 & Lab 05.	**Session 3 (Part A):** Root Context Maps & Memory Hygiene.	**Lab 05:** `context_compiler.py` (`REQ-0501..0506`).
Module 6 (Slides 24–29)	**Week 6:** Lec 6A (`SKILL.md`, `chub`, Meta-MCP & `SkillsBench`) + Lab 6B.	**Day 2, 11:00–13:00:** Skills, Code Mode, Demo 6 & Lab 06.	**Session 3 (Part B):** Curated Skills & Meta-MCP Architecture.	**Lab 06:** `skill_and_mcp_harness.py` (`REQ-0601..0606`).
Module 7 (Slides 30–34)	**Week 7:** Lec 7A (Subagent Firewalls, Council & `jj`) + Lab 7B.	**Day 2, 14:00–16:00:** PR Triage, `llm-council`, Demo 7 & Lab 07.	**Session 4 (Part A):** Multi-Agent Firewalls & Jujutsu VCS.	**Lab 07:** `subagent_council.py` (`REQ-0701..0706`).
Module 8 (Slides 35–42)	**Week 8:** Lec 8A (`Omnigent`, `autoresearch` & RSI) + Lab 8B + Capstone.	**Day 2, 16:00–18:00:** Governance, Ratchets, Demo 8 & Lab 08.	**Session 4 (Part B):** Meta-Harnesses, Linters & Program Design.	**Lab 08:** `autore search_harness.py` (`REQ-0801..0806`).

Table 1: Three turnkey pacing schedules mapping the 8 modules, 42 slides, 8 projector demos, and 8 hands-on labs to university, bootcamp, and staff-workshop formats.

2 Phase I Lecture Delivery Playbook (Modules 1–4, Slides 01–19): Live Demos, Chart Walkthroughs & Socratic Prompts

2.1 Module 1 (Slides 01–07): Language, Code as a Conceptual Model, The Visual Evolution Timeline & The 17-Day Model Pace

Opening Lecture Narrative & Minute-by-Minute Pacing (Slides 01–02, 15 Minutes). When opening Module 1, your goal in the first ten minutes is to disarm two opposite extremes in the room: skeptics who view coding agents as glorified autocomplete, and enthusiasts who believe engineers no longer need to read code. Spend 5 minutes on Slide 01 orienting students to the dual-track repository (`agy` and `claude`). On Slide 02 (10 minutes), introduce the conceptual foundation of why natural language produces executable code at all. Contrast the two roles of language developed by Ludwig Wittgenstein: formal logic as a strict picture of facts (which maps to compilers, static type checkers, and deterministic unit tests) versus everyday language as an active collaborative toolbox used between a builder and an assistant to coordinate actions. Pair this immediately with Unmesh Joshi and Martin Fowler's insight that source code is not merely syntax for a CPU, but an externalized model of understanding. When engineers passively accept "vibe-coded" output without maintaining domain vocabulary, the codebase accumulates cognitive debt through synonym drift (`client_id`, `cust_acct`, `user_id`) and silent floating-point currency bugs.

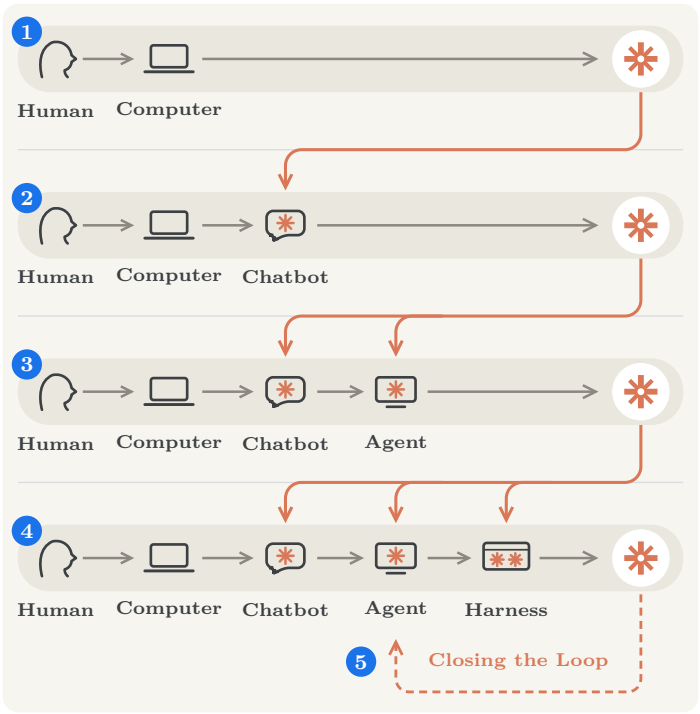
Teaching the Anthropic-Style Visual Evolution Timeline on Slide 03 & The Six Eras on Slide 04 (12 Minutes). When you advance to Slide 03 (reproduced for instructor reference in Figure 2), let the visual diagram speak before reading any bullet points. Point to the four horizontal rounded tracks on the left and ask the class to trace what happens to the orange feedback arrow leaving the `AI MODEL` starburst badge at the end of each row:

1. **Row 1 (2021–2023, Manual Coding):** A human sitting at a laptop (`Human - Computer`) writes code manually (assisted only by line-level `Tab` completion) to train the first frontier AI models.
2. **Row 2 (2023–2024, Chatbots & Tool Protocols):** That first generation of models feeds back into the workflow as a `Chatbot` between the laptop and the model, while `Function Calling` and `MCP` standardize tool invocation.
3. **Row 3 (2025, Coding Agents & Vibe Coding):** The second generation of models powers both the `Chatbot` and a repository-aware coding `Agent` (`claude` and `agy`) capable of editing files and running tests under `CLAUDE.md`, `GEMINI.md`, and `SKILL.md`.
4. **Row 4 (2026 & Closing the Loop, Autonomous Harnesses):** Today's models power the `Chatbot`, the `Agent`, and a multi-agent `Harness` (`git worktree`, colocated `jj`, `Temporal` durable replay, and `Omnigent` meta-harnesses), while the dashed recursive loop at the bottom (*Closing the Loop*) shows bounded verification loops (`autoresearch` and kernel optimization) accelerating the next generation of models. Follow this on Slide 04 with the six architectural eras from 2022 to 2026.

How to Walk Students Through the Three Model-Pace Charts on Slide 05 (10 Minutes). When you advance to Slide 05, pause and ask the class how often they think a frontier model changes in production. Then walk them systematically through the three full-height empirical release-pace panels ($N = 143$ notable releases and 57 flagships across eight global lab groups):

1. **Panel 1 (8-Lab Swimlane Dot-Plot, $N = 143$ Releases / 57 Flagships):** Point out the horizontal swimlanes across OpenAI (38 releases), Google (23), Chinese labs (20), Anthropic (17), DeepSeek (13), Meta (12), Other (11), and xAI (9). Have students observe visually how sparse the dots are in 2023 compared to the dense cluster of solid flagship markers in 2025 and 2026.
2. **Panel 2 (16-Quarter Release Volume Surge, `4/qtr` → `22/qtr`):** Trace the stacked quarterly bars showing total notable releases surging 5.5× from 4 releases per quarter in early 2023 to 9 per quarter in

Figure 2.1 Blueprint: Slide 03 Visual Evolution Timeline & Instructor Board Cues



1. 2021–2023 (Row 1 Cue): Point out the single gray arrow: humans wrote all code on laptops to train the first models.

2. 2023–2024 (Row 2 Cue): Trace the first orange feedback arrow: the model powers a `Chatbot` + `MCP` tool protocols.

3. 2025 (Row 3 Cue): Trace the two-branch feedback arrow: RLVR models power both `Chatbot` and coding `Agent` (`claude` & `agy`).

4. TODAY 2026 (Row 4 Cue): Trace the three-branch feedback arrow into `Harness` (`git worktree`, `jj`, `Temporal`, `Omnigent`).

5. 2026+ (Dashed Loop Cue): Highlight the dashed recursive loop (`autoresearch`, kernel optimization) accelerating the next model generation.

Figure 2: Instructor visual walkthrough for Slide 03 showing how each generation of frontier AI models feeds back into the developer toolchain (`Chatbot` → `Agent` → `Harness` → `Closing the Loop`).

2024 and peaking at 22 releases (including 7 flagships) in a single quarter.

3. **Panel 3 (Inter-Flagship Interval Collapse, `237d` → `18–21d`):** Highlight the steep downward slope: the longest gap between a lab's own flagships dropped from 237 days at OpenAI down to 18–21 days in mid-2026, compressing the industry-wide flagship gap from 37 days down to just **17 days**. Emphasize the takeaway for the course: because the frontier model behind an agent changes every 17 days, prompt tricks tied to one model snapshot decay in weeks, whereas a well-engineered harness endures across model generations.

Teaching the 4-Pillar Skills Map, Earned Autonomy & Projector Demo 1 (Slides 06–07, 13 Minutes). On Slide 06, connect Andrew Ng's four-pillar AI Engineering Skills Map to the Ladder of Earned Autonomy. Emphasize Pillar 2 (Software Engineering Fundamentals): novices who prompt an agent without specifying latency, consistency, reliability, and security constraints force the LLM to guess architectural trade-offs silently. Before transitioning to Lab 01 on Slide 07, switch the classroom projector to your terminal and run **Projector Demo 1**:

Projector Demo 1 (Slide 07): Unsteered Floating-Point Vibe Prompt vs. SE-Steered Prompt & Hard Floors

```
# 1. Show flawed starter accepting synonym drift & IEEE-754 float money:
$ python3 -c "print(0.1 + 0.2 == 0.3, round(19.99 * 100))"
# False 1999  <-- Ask class: What happens to ledger balances with floats?

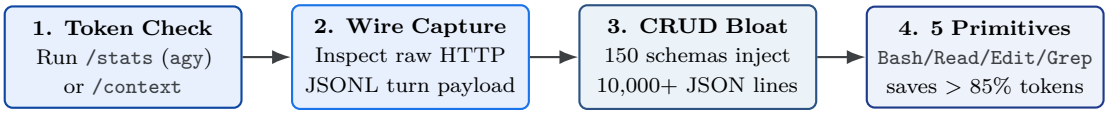
# 2. Run Lab 01 self-diagnose enforcing MoneyCents & Hard Floors:
$ ./labs/lab_01_vibe_coding_and_earned_autonomy/self_diagnose.sh
# EarnedAutonomyGate blocks 'rm -rf /' & 'adversarial_tests/' in auto mode.
```

2.2 Module 2 (Slides 08–11): Inside the Agentic Loop, agy & claude 101, and Wire-Trace Inspection

Lecture Narrative & The "Swiss Army Knife" Tool Principle (Slides 08–11, 45 Minutes). Begin Module 2 on Slide 08 by demystifying what a coding agent actually is: a three-layer stack consisting of the stateless Model API at Layer 1, File/Shell Toolkits and MCP servers at Layer 2, and the Agent Harness at Layer 3. Contrast a minimal 200-line Python agent loop against a Wireshark-style HTTP/JSONL wire capture of a production session. Students are invariably shocked when you point out the empirical finding from Stanford's modern software developer curriculum: mounting 150 granular CRUD MCP tools injects over **10,000 lines of JSON schema definitions** into every turn for a 600-line chat conversation ($> 16\times$ schema-to-conversation bloat). Explain why production harnesses consolidate tools into 5 to 8 composable "Swiss Army Knife" primitives (`Bash`, `Read`, `Write`, `Edit`, `Grep`, `LSP`), leaving 88% of the context window free for actual reasoning.

Comparing Google Antigravity (agy) and Claude Code (claude) Surfaces (Slides 09–10) & Projector Demo 2. On Slide 09, walk through Google Antigravity 101 across its three surfaces (Mission Control Agent Manager, the IDE editor with `Cmd+L` and Tab-to-Jump, and the `agy` CLI) and its three verifiable Artifacts (`Task List`, `Implementation Plan` with inline review comments, and `Walkthrough` run receipts with live `/browser` screenshots). On Slide 10, present the parallel mechanics in Anthropic Claude Code 101 (`@file` context pinning, `!` shell mode, `Shift+Tab` permission cycling across Normal, Auto-Accept, and Plan Mode, plus `/context`, `/compact`, `/clear`, `/rewind`, and `/btw`). Then run **Projector Demo 2**:

Projector Demo 2 (Slide 11): Live JSONL Wire-Trace Inspection & Tool Schema Bloat vs. Core Primitives




```
# 1. Show live session token breakdown on the classroom projector:
#    In agy: run '/stats'       |   In claude: run '/context'
# 2. Run Lab 02 wire-trace inspector comparing 150 CRUD vs. 5 core tools:
$ ./labs/lab_02_agent_loop_and_harness_wire_trace/self_diagnose.sh
```

2.3 Module 3 (Slides 12–15): Directing the Daily Workflow (RePPIT), Orthogonal Proposals & Context Reset

Calibrating Ceremony to Blast Radius & Defeating Silent Assumption Syndrome (Slide 12). Students often ask whether they must write a formal plan for a one-line typo fix. Use Slide 12 to establish the Task Complexity vs. Workflow Rigor spectrum: a 1-file copy tweak uses a direct prompt; a 2–3 file bugfix uses a single Plan Mode pass; a medium 4–12 file feature uses the full five-step `RePPIT` loop; and a cross-service migration uses multi-session subagents. Emphasize how to defeat **Silent Assumption Syndrome**—where RLHF-trained models silently guess authentication schemes, retry policies, or database isolation levels rather than bothering the user—by inverting control via `/grill-me` (in `agy`) or Plan Mode interviewing (in `claude`).

Teaching the 5-Step `RePPIT` Loop & The Post-Proposal Context Reset Rule (Slides 13–15). On Slide 13, walk carefully through all five steps of `RePPIT` (`Research -> Propose -> Plan -> Implement -> Verify`). Pause at Step 2 (`Propose`) to highlight one of the highest-leverage habits in the entire course: after forcing the agent to propose at least **two orthogonal designs** (for example, a stateful database idempotency store versus a stateless HMAC-signed retry token) and choosing one, the engineer *must* execute a **Post-Proposal Context Reset** before Step 3 (`Plan`). If 3,000 tokens of the rejected proposal remain in the chat scroll during implementation, attention bleed causes the model to accidentally blend abstractions

from both designs. On Slide 14, compare inline plan co-editing (`Implementation Plan` comments in `agy` vs. `Ctrl+G` in `claude`) and checkpoint time-travel (`Undo changes` vs. `/rewind` via `Esc+Esc`), then execute **Projector Demo 3**:

Projector Demo 3 (Slides 14–15): Socratic `/grill-me`, Orthogonal Proposals & Post-Proposal Reset

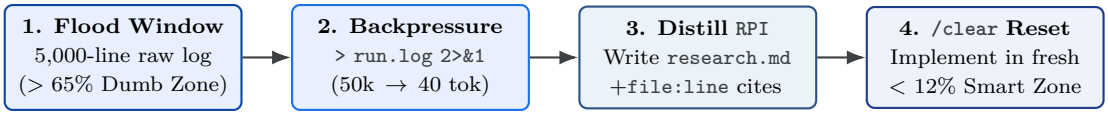
```
# 1. Prompt the agent with an ambiguous feature request in Plan Mode:
#    "Add webhook retries to payment processor. Run /grill-me first."
# 2. Show the agent asking 4 clarifying questions (jitter, DLQ, keys).
# 3. Request 2 orthogonal proposals, purge Rejected Option, and verify:
$ ./labs/lab_03_reppit_workflow_and_reflection/self_diagnose.sh
```

2.4 Module 4 (Slides 16–19): Context Window Physics, The 40% "Dumb Zone" & Frequent Intentional Compaction

How to Explain the Context Window Physics Curve on Slide 16 & The Sawtooth Compaction Chart on Slide 18. Slide 16 is the conceptual pivot of the entire course. Write the fundamental stateless equation on the whiteboard: NextAction = LLM(ContextWindow). Walk students through the utilization curve on Slide 16: between 0% and 40% utilization (the **Smart Zone**), transformer attention remains sharp across every system rule and type signature. Once unfiltered `grep` dumps, 5,000-line test logs, and multi-turn chat arguments push utilization past the **40% threshold**, attention dilutes across competing historical tokens and the agent enters the **Dumb Zone** (Context Rot). On Slide 17, show how to apply **Context Backpressure** by redirecting verbose test and training output to `run.log` (`> run.log 2>&1`) and grepping only summary metric lines. Then on Slide 18, walk through the two side-by-side diagrams:

- **Left Chart (Sawtooth Utilization via Frequent Intentional Compaction, `RPI`):** Contrast the naive monotonically rising trajectory (which crosses 40% mid-task and stays trapped in the Dumb Zone until emergency 95% auto-compaction) against the intentional sawtooth wave: distill exploration into `research.md` with exact `file:line` citations → run `/clear` → draft `plan.md` → run `/clear` → implement inside a fresh $< 12\%$ Smart Zone window.
- **Right Chart (The Human Attention Leverage Pyramid):** Explain why human review belongs on `research.md` and `plan.md`: 1 bad line in `research.md` amplifies into $\sim$ 1,000 lines of broken code (1000× defect amplification), and 1 bad line in `plan.md` amplifies into $\sim$ 100 wasted lines (100×), whereas reviewing raw generated code is 1× leverage.

Projector Demo 4 (Slides 17–19): Watching Context Rot Past 40%, Log Backpressure & `RPI` Compaction




```
# Demonstrate autoresearch log backpressure live in the terminal:
$ ./self_diagnose_all.sh > /tmp/run.log 2>&1 && tail -n 5 /tmp/run.log
# Then run Lab 04 context profiler and RPI compactor verification:
$ ./labs/lab_04_context_physics_and_rpi_compaction/self_diagnose.sh
```

Synthesizing Phase I Across Modules 1–4 Before Entering Lab Workshops. Before releasing students to their Phase I lab stations, review the complete Phase I instructor delivery arc in Figure 3 and the slide-by-slide pacing matrix in Table 2. Remind students that every habit practiced manually in

Phase I—Ubiquitous Language, tool minimalism, `RePPIT` plan verification, and keeping utilization below 40%—becomes an automated software mechanism in Phase II.

Figure 2.2 Blueprint: Phase I Lecture Pacing, Projector Demos 1–4 & Socratic Checkpoints

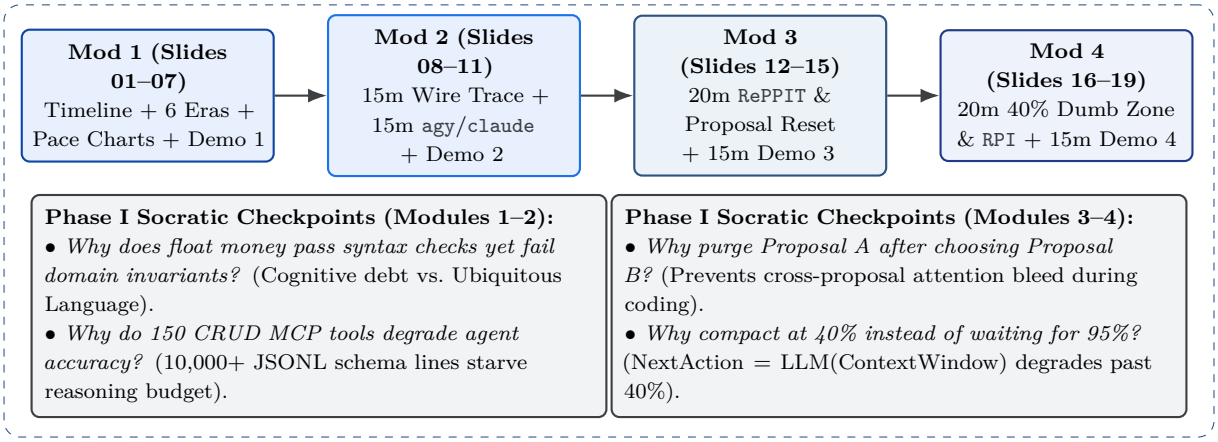


Figure 3: Phase I Instructor Delivery Blueprint mapping Modules 1–4 (Slides 01–19) across lecture segments, live projector demos, and Socratic classroom interventions.

Slides	Core Lecture Concept & Visuals	Live Projector Demo / Action	Socratic Prompt & Misconception Correction
Slides 01–07 (Module 1)	Wittgenstein's 2 theories; Fowler's Cognitive Debt; Visual Evolution Timeline (Slide 03); Six Eras (Slide 04); 3 SearchIntel charts (Slide 05); 4-Pillar Map.	**Demo 1:** Run float rounding failure vs. `MoneyCents` & `Earned AutonomyGate` hard-floor block in Lab 01.	*"If the frontier model changes every 17 days, where should team engineering effort go?"* → Into model-agnostic harnesses and deterministic tests, not brittle prompt hacks.
Slides 08–11 (Module 2)	3-Layer `aisuite` stack; 200-line loop vs. JSONL wire trace; `agy` Artifacts vs. `claude` modes.	**Demo 2:** Run `/stats` or `/context`; inspect 150-tool CRUD bloat vs. 5 Swiss-Army tools in Lab 02.	*"Why not expose every REST endpoint as an MCP tool?"* → 150 schemas inject 10,000+ lines per turn; composable CLI primitives (`Bash`, `Read`, `Edit`, `Grep`) save $> 85\%$.
Slides 12–15 (Module 3)	Complexity vs. Rigor; `/grill-me`; 5-step `RePPIT`; Post-Proposal Reset; `Out of Scope` guardrails.	**Demo 3:** Trigger `/grill-me` on an ambiguous prompt; generate 2 proposals and purge the rejected option in Lab 03.	*"Why did the agent refactor three unrelated files?"* → Because the plan lacked an explicit `Out of Scope / Do NOT Touch` boundary and kept rejected proposal tokens.
Slides 16–19 (Module 4)	NextAction = LLM(Window); 40% Dumb Zone curve; `> run.log 2>&1`; Sawtooth `RPI` & Leverage Pyramid.	**Demo 4:** Redirect test stdout to `run.log`; compact noisy turns into line-cited `research.md` and `plan.md` in Lab 04.	*"My model has a 1M-token window; why use `/clear` at 40%?"* → Effective attention degrades long before capacity limits as noisy logs dilute instruction weights.

Table 2: Phase I Slide-by-Slide Instructor Pacing, Live Projector Checkpoints, and Socratic Interventions (Slides 01–19).

3 Phase II Lecture Delivery Playbook (Modules 5–8, Slides 20–42): Live Demos, Chart Walkthroughs & Socratic Prompts

3.1 Module 5 (Slides 20–23): The 2×2 Cybernetic Control Matrix, The 60-Line Map Rule & Cross-Session Auto-Memory

Opening Phase II with the 2×2 Cybernetic Control Matrix (Slide 20, 12 Minutes). When transitioning the classroom into Phase II on Slide 20, announce that students are no longer merely steering a chat session—they are now systems architects building the cybernetic environment around the model (Agent = Model + Harness). Draw Birgitta Böckeler and Kief Morris's 2×2 Cybernetic Control Matrix on the board: cross **Timing** (*Feedforward Guides* before action versus *Feedback Sensors* after action) with **Mechanism** (*Computational CPU Checks* versus *Inferential LLM Checks*). Have students classify every tool they will build over the next four modules into these four quadrants:

- **Quadrant 1 (Computational Guides):** Container sandboxes, static type systems, and `PreToolUse` Exit-Code-2 hard floors.
- **Quadrant 2 (Computational Sensors):** Deterministic unit tests (`self_diagnose.sh`), immutable evaluators (`prepare.py`), and structural AST linters.
- **Quadrant 3 (Inferential Guides):** Root context maps (`CLAUDE.md`, `GEMINI.md`), progressive-disclosure skills (`SKILL.md`), and curated API docs (`chub`).
- **Quadrant 4 (Inferential Sensors):** Multi-agent pull-request reviewers (`code-reviewer`) and anonymized cross-model peer councils (`llm-council`).

The 150-Instruction Compliance Cliff, The 60-Line Map Rule & Projector Demo 5 (Slides 21–23, 33 Minutes). On Slide 21, distinguish the three context primitives—ephemeral Prompts, standing or glob-scoped Rules, and lazy-loaded Skills—and walk students through the instruction-compliance curve. Point out how compliance remains near 98% up to roughly 60 lines, hits a steep compliance cliff past **150 simultaneous instructions**, and degrades across *all* rules when teams dump 400 lines of prose into a monolithic root file. Teach the **Minimalist 60-Line Map Rule**: treat root `CLAUDE.md` and `GEMINI.md` (with `AGENTS.md` symlinked) as an L1 CPU cache containing only exact build/test/lint commands, a concise directory map, and lazy `@docs/*.md` pointers. On Slide 22, contrast human-authored root maps against cross-session auto-memory (`MEMORY.md` in Claude Code, shared across git worktrees and capped at 200 startup lines, versus `KNOWLEDGE.md` in Google Antigravity managed via `/memory`), stressing strict provenance discipline (`#direct` only for explicit user rules; `#commit`, `#time`, and `#session` for agent observations). Conclude with **Projector Demo 5**:

Projector Demo 5 (Slide 23): Refactoring a 350-Line Instruction Dump into a 45-Line Map + `@docs/` Imports

```
# 1. Inspect the 350-line monolithic dump in starter/ of Lab 05.
# 2. Show the live 45-line root CLAUDE.md / GEMINI.md with @docs/ imports:
$ wc -l CLAUDE.md GEMINI.md && head -n 25 CLAUDE.md
# 3. Run Lab 05 context compiler and provenance-tagged memory verifier:
$ ./labs/lab_05_context_primitives_and_memory/self_diagnose.sh
```

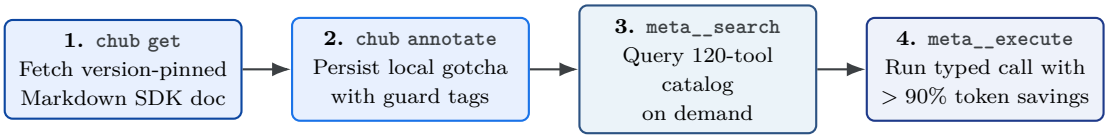
3.2 Module 6 (Slides 24–29): Progressive-Disclosure Skills, chub Grounding, Meta-MCP Code Mode & SkillsBench

Teaching Progressive Disclosure, chub Self-Annotating Docs & Meta-MCP Code Mode (Slides 24–26, 20 Minutes). On Slide 24, explain how the open agentskills.io standard scales to dozens of procedural workflows via three-level progressive disclosure: Level 1 loads only YAML frontmatter (name and description with explicit Use when and Don't use for boundaries, $\sim$ 100 tokens) at startup; Level 2 loads the $\leq$ 500-line SKILL.md body on trigger; and Level 3 keeps references/ and scripts/ on disk at zero token cost until needed. On Slide 25, walk through Andrew Ng's context-hub (chub) four-stage loop: fetching version-pinned Markdown docs via chub get, recording discovered runtime gotchas locally via chub annotate, reloading notes inside untrusted-input guards via --with-annotations, and sending quality ratings upstream via chub feedback. On Slide 26, present the four rules of ergonomic MCP tool design and show why **Meta-MCP "Code Mode"**—exposing only search and execute meta-tools instead of 150 raw CRUD schemas—slashes tool token overhead by $>$ 90%.

How to Explain the Four Spacious SkillsBench Empirical Charts Across Slides 27 & 28 (15 Minutes). Slides 27 and 28 present four empirical plots from the SkillsBench study across 84 tasks, 11 domains, and 7,308 agent trajectories. Guide students through the two slides in sequence:

1. **Slide 27 Left Chart (Pass Rate by Harness & Condition):** Point out that human-curated skills boost pass rates by **+16.2 percentage points** on average (lifting Claude Opus 4.5 by +23.3 points and Gemini 3 Flash to 48.7%), whereas **self-generated skills average −1.3 percentage points** (and drop GPT-5.2 on Codex by −5.6 points) because models cannot reliably author the procedural domain knowledge they lack. Highlight the striking economic result: smaller Haiku 4.5 *with* curated skills (27.7%) outperforms frontier Opus 4.5 *without* skills (22.0%).
2. **Slide 27 Right Chart (Domain Heterogeneity Across 11 Domains):** Contrast domains with low pretraining coverage—Healthcare (**+51.9 pp**), Manufacturing (**+41.9 pp**), Cybersecurity (**+23.2 pp**), and Natural Science (**+21.9 pp**)—against general Software Engineering (**+4.5 pp**), where base models already hold strong priors.
3. **Slide 28 Left Chart (Skill Quantity & Length Sweet Spot):** Show that mounting **2 to 3 focused skills** peaks at **+18.6 pp**, whereas mounting **4+ overlapping skills** drops gain to +5.9 pp from routing confusion, and comprehensive encyclopedic manuals hurt performance by **−2.9 pp**.
4. **Slide 28 Right Chart (Failure Reduction & Cost-Performance Pareto):** Point out how curated skills cut Quality-Below-Threshold failures by 30.8% and Incomplete Solutions by 35.8%, placing Gemini 3 Flash + Skills on the Pareto frontier at $0.57 per trial versus $1.06 for Pro.

Projector Demo 6 (Slide 29): chub get + chub annotate & Meta-MCP search+execute (>90% Savings)




```
# Run Lab 06 to verify 3-level skills, chub notes, and Meta-MCP Code Mode:
$ ./labs/lab_06_skills_chub_and_meta_mcp_code_mode/self_diagnose.sh
# MetaMcpCodeModeHarness compresses 120 CRUD schemas by >90%.
```

3.3 Module 7 (Slides 30–34): Context-Firewall Subagents, PR Triage, llm-council & Jujutsu (jj) VCS

Teaching Subagents as Context Firewalls, Multi-Agent PR Triage & llm-council (Slides 30–32, 20 Minutes). On Slide 30, warn students against the widespread anti-pattern of anthropomorphizing subagents into corporate roleplay personas ("Frontend Bob" and "PM Alice"). State the physical law clearly: **subagents exist as Context Firewalls**. A read-only child subagent (`code-explorer`) can burn 50,000 tokens searching 25 files inside its own isolated window and return a 500-token cited summary to the parent (100:1 compression), keeping the parent inside the Smart Zone. On Slide 31, trace the seven-phase `feature-dev` pipeline and the five-lens pull-request triage (Security, Performance, Architecture, Style, and Edge Cases) gated at confidence ≥ 80 into `Must Fix`, `Recommended`, and `Consider`. On Slide 32, explain Andrej Karpathy's three-stage `llm-council` (Stage 1 parallel opinions → Stage 2 anonymized peer ballots `Response A/B/C` → Stage 3 Chairman Borda synthesis) to eliminate single-model self-preference bias.

Contrasting `git worktree` Provisioning Cost vs. Colocated Jujutsu (jj) Integration Cost (Slides 33–34). On Slide 33, walk through the multi-agent version control dichotomy. Native `git worktree` (used by Antigravity Mission Control and `claude --worktree`) shares the `.git` object store to slash cold-start **Provisioning Cost**. However, when 5 parallel agents continuously rewrite stacked commits, Git's stateful index and abortive rebase freeze autonomous loops. Show how colocating Jujutsu (`jj git init --colocate` and `jj workspace add`) slashes **Integration Cost** via three breakthroughs: working-copy-as-a-commit (`@`) with stable Change IDs, first-class **Conflict Algebra** ($\Delta M = \text{Tree}(M) - \text{AutoMerge}(P_1, P_2)$) so rebases never abort mid-loop, and transactional `jj op log` / `jj undo` recovery. Then run **Projector Demo 7**:

Projector Demo 7 (Slide 34): Context-Firewall Compression, Anonymized `llm-council` & Worktree Isolation

```
# Run Lab 07 for 50k->500 subagent firewall, >=80 triage & Borda council:
$ ./labs/lab_07_subagent_firewalls_and_peer_council/self_diagnose.sh
```

3.4 Module 8 (Slides 35–42): 4-Tier Governance, Omnigent, Remediation Linters, Autoresearch & RSI Charts

Walking Through 4-Tier Governance (Slide 35) and the 4-Stage `Omnigent` Meta-Harness Architecture (Slide 36). Open Module 8 on Slide 35 by explaining how four-tier governance (Exit-Code-2 hard floors, stateful contextual policies, durable execution replay via `Temporal/DBOS/Restate`, and egress-proxy sandboxes) protects long-horizon runs. Then on Slide 36, walk through the four-stage **Databricks `Omnigent` Meta-Harness** architecture:

1. **Stage 1 (Heterogeneous Agents):** Unifies CLI harnesses (`Claude Code`, `Codex`, `OpenCode`, `Antigravity`, `Pi`), declarative YAML subagent graphs, and custom SDK agents.
2. **Stage 2 (`Omnigent` Runner):** Wraps execution in container sandboxes with automatic retry reliability and **egress-proxy credential injection** so raw GitHub or cloud tokens never enter the container environment.
3. **Stage 3 (`Omnigent` Server):** Centralizes six core services—`History`, `Catalog`, `Policies` (such as escalating `git push` to human approval after `npm install` or pausing every $100 of spend), `MCPs`, `Artifacts`, and `Skills`—backed by PostgreSQL, MLflow tracing, and automated context optimizers (`GEPA`, `MemEx`, `RLM`).
4. **Stage 4 (Five Multiplayer Surfaces):** Streams live sessions simultaneously across Terminal UI, Web UI (with Google Docs-style inline diff comments), Native Desktop App, Mobile UI, and REST API.

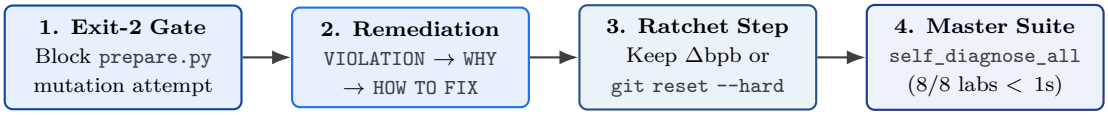
Teaching Remediation Linters, Karpathy's 83-Experiment `autoresearch` Staircase & The Dark

Factory Curve (Slides 37–39, 18 Minutes). On Slide 37, contrast cryptic human linter errors (`E901 Forbidden import`) against OpenAI's three-part remediation linter format (`[VIOLATION] -> [WHY] -> [HOW TO FIX]`), which gives the LLM the exact replacement import in one turn. On Slide 38, walk through Andrej Karpathy's three-file `autoresearch` harness (`prepare.py` immutable evaluator locked by Exit Code 2, `train.py` single mutable file, and `program.md` research org code) and trace the **83-experiment `progress.png` step-down staircase**: point out the 15 kept commits stepping validation bits-per-byte down from the **0.9979** baseline through batch halving, depth-9 aspect-57 (**0.9846**), short 1/8th sliding windows (**0.9797**), and RoPE 200k scaling to **0.9773**, while 68 gray failed or over-complex trials above the staircase were immediately discarded via `git reset --hard` under the **Simplicity Criterion**. Then on Slide 39, contrast scalar kernel ratchets against full-stack products using the **Dark Factory Velocity Curve**: explain why unattended "lights-off" factories collapse around Week 8 due to the **RLVR Reward Horizon Gap** ($t = 0$ test passing vs. $t = 6$ month architectural maintainability), and why humans must own four-phase front-loaded alignment (`Product -> Architecture -> Program Design [Types, Interfaces, Call-Trees] -> Vertical Slices`).

How to Explain Anthropic's Four Recursive Self-Improvement (RSI) Charts Across Slides 40 & 41 + Projector Demo 8 (15 Minutes). Conclude the lecture on Slides 40 and 41 by walking through the four spacious empirical charts from Anthropic Institute's study on Recursive Self-Improvement:

- **Slide 40 Left Chart (Code Contributed per Person by Quarter, `1.0x` → `8.0x`):** Show how quarterly LOC per engineer stayed flat at 1.0× from 2021 through 2024, then inflected sharply to 2.5× in Q4 2025, 5.8× in Q1 2026, and **8.0**× in Q2 2026.
- **Slide 40 Right Chart (`Claude Code` Session Success Rate by Task Difficulty, `26%` → `91%`):** Trace the four difficulty curves from September 2025 to September 2026: open-ended ML research tasks climbed from 26% (and an 11% dip in October 2025) to 71% in April 2026, 76% in May 2026, and **91%** by September 2026, converging with trivial (89%), routine (92%), and substantial (92%) tasks.
- **Slide 41 Left Chart (Blind Peer-Reviewed Researcher Next-Step Evaluation, $n = 129$ Dead-Ends):** On 129 historical research dead-ends where a human researcher made a suboptimal move, blind peer review showed the frontier model selecting a strictly better next step **64%** of the time (**73%** including ties), climbing from 22% with Haiku 3 and surpassing the 50% human baseline toward the 90% ceiling.
- **Slide 41 Right Chart (METR Horizon Doubling to `16h` & `52x` Kernel Loop):** Pair the 4-month task-horizon doubling curve (30s → 4m → 45m → 16h) with the **52× CPU kernel optimization speedup** achieved in an 8-hour autonomous loop versus a 4× human baseline, guarded by Out-of-Tree Scenario Holdouts.

Projector Demo 8 (Slide 42): Exit-Code-2 Hard Floor, Remediation Linter, `autoresearch` Ratchet & Full Course Check




```
# Run Lab 08 and then the full 8-lab master verification suite live:
$ ./labs/lab_08_governance_linters_and_autoresearch_ratchet/self_diagnose.sh
$ ./self_diagnose_all.sh
```

Synthesizing Phase II Across Modules 5–8 Before Capstone & Lab Assessment. Use Figure 4 and Table 3 to review how the four layers of Harness Engineering connect into a single cybernetic architecture across Slides 20 through 42.

Figure 3.1 Blueprint: Phase II Lecture Pacing, Empirical Chart Walkthroughs & Demos 5–8

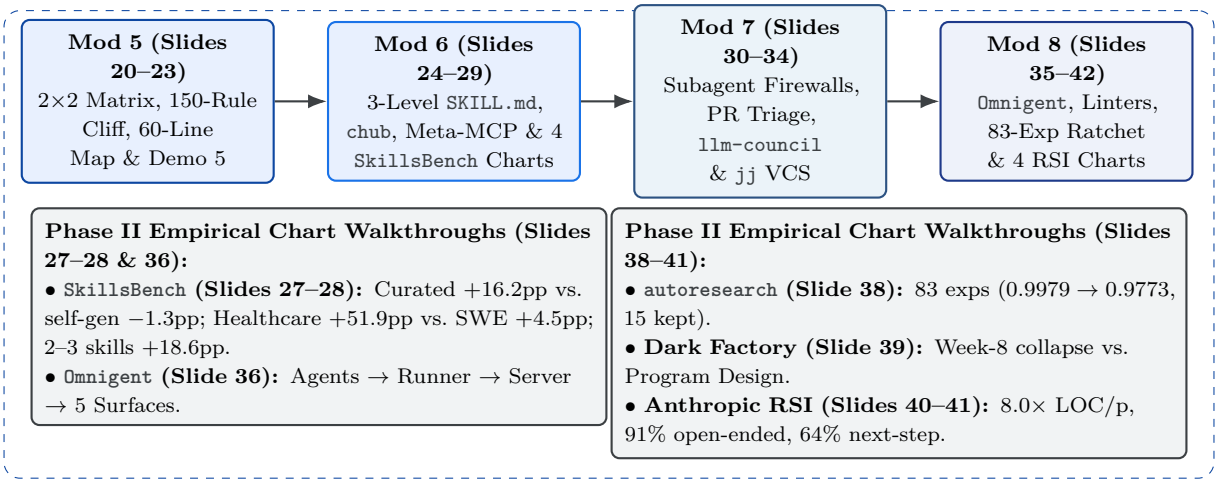


Figure 4: Phase II Instructor Delivery Blueprint mapping Modules 5–8 (Slides 20–42) across the four Harness Engineering layers, empirical chart walkthroughs, and Projector Demos 5–8.

Slides	Quantitative Charts & Diagrams	Live Projector Demo / Action	Socratic Prompt & Misconception Correction
Slides 20–23 (Module 5)	2×2 Cybernetic Control Matrix; 150-Instruction Compliance Cliff; 60-Line Root Map & Auto-Memory.	**Demo 5:** Refactor a 350-line rule dump into a 45-line `CLAUDE.md/GEMINI.md` map + `@docs/` in Lab 05.	*"Why not let the agent tag its own codebase observations as `#direct`?"* → Speculative agent guesses masquerading as user rules permanently poison future sessions.
Slides 24–29 (Module 6)	3-Level `SKILL.md`; `chub` loop; Meta-MCP token bars; 4 spacious `SkillsBench` charts ($N = 7{,}308$ trajectories).	**Demo 6:** Run `chub annotate` and Meta-MCP `search+execute` (> 90% schema savings) in Lab 06.	*"Why did self-generated skills score −1.3pp on `SkillsBench`?"* → Models cannot author procedural domain knowledge they lack; human curation (+16.2pp) is essential.
Slides 30–34 (Module 7)	Context-Firewall (50k → 500 tok); 7-phase `feature-dev`; 5-lens PR triage; `llm-council`; `git worktree` vs. `jj`.	**Demo 7:** Run read-only subagent firewall, ≥ 80 PR triage, and Borda council ranking in Lab 07.	*"Why does Jujutsu (`jj`) outperform `git rebase` for parallel agents?"* → `jj` stores conflicts as algebraic trees inside commits so rebases never abort mid-loop.
Slides 35–42 (Module 8)	4-stage `Omnigent`; 83-exp `autoresearch` (0.9979 → 0.9773); Dark Factory curve; 4 Anthropic RSI charts (8.0×, 91%, 64%, 52×).	**Demo 8:** Trigger Exit-Code-2 lock on `prepare.py`, remediation linter, and ratchet step in Lab 08.	*"If `autoresearch` works overnight on ML kernels, why do dark software factories fail at Week 8?"* → Multi-module software lacks a scalar $t = 0$ metric; humans must own Program Design.

Table 3: Phase II Slide-by-Slide Instructor Pacing, Empirical Chart Walkthroughs, Live Demos, and Socratic Interventions (Slides 20–42).

4 Complete Lab Facilitation Guide, Instructor Answer Key & TA Debugging Playbook (Labs 01–08)

4.1 How to Facilitate the 90-Minute Hands-On Lab Workshops

Running High-Agency Lab Workshops Without Giving Away the Code. During the supervised lab sessions, teaching assistants (TAs) and lab facilitators should resist the urge to debug student code directly at the keyboard. Every lab in the repository is structured around four directories: `starter/` (containing an intentionally flawed anti-pattern implementation), `expected_output/` (the instructor reference solution), `adversarial_tests/` (locked read-only grading assertions testing `REQ-0X01` through `REQ-0X06`), and `work/` (where the student steers their coding agent to build the solution). When a student's implementation fails `self_diagnose.sh work`, ask the student to show you their `research.md` or `Implementation Plan` first. Nine times out of ten, a failing unit test traces back to a missing invariant in the student's prompt or plan rather than a Python syntax mystery (see Figure 5 below).

Figure 4.1 Blueprint: 4-Stage Lab Autograding Pipeline & Anti-Reward-Hacking Lock

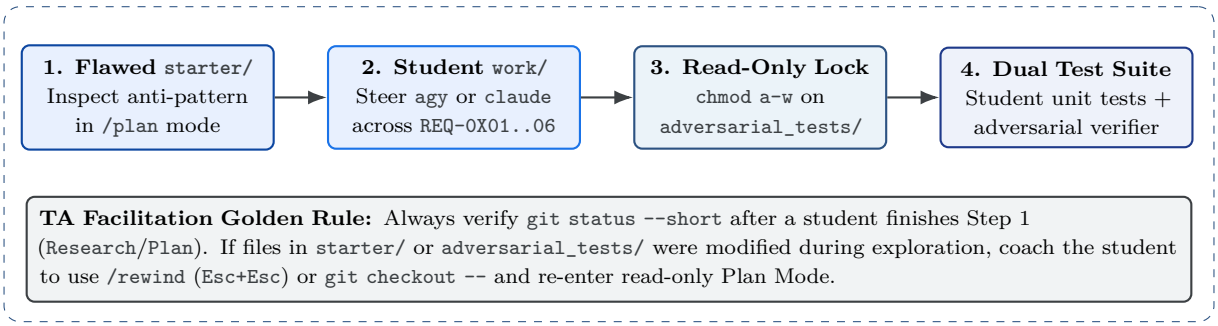


Figure 5: The four-stage deterministic lab verification pipeline enforced by each lab's `self_diagnose.sh` runner across all eight course modules.

4.2 Phase I Lab Answer Keys, Everyday Metaphors & 3-Rung TA Hint Ladders (Labs 01–04)

Lab 01 Answer Key: `prompt_steerer.py` (`labs/lab_01_vibe_coding_and_earned_autonomy`).

- **Everyday Metaphor (`Known` → `Target`) & `TaskFlow` Framing:** Naive vibe coding is like telling a contractor "build me a fast checkout for `TaskFlow` Pro" without electrical blueprints or circuit breakers; **Engineering Steering & Earned Autonomy** provides the 5-dimensional engineering blueprint plus hard-floor safety interlocks that cannot be bypassed even in `auto` mode.
- **Intentional Flaws in `starter/vibe_prompt_and_autonomy_blob.py`:** Accepts drifted synonyms (`client_id`, `cust_acct`, `user_id`, `tx_amt`) and IEEE-754 `float` currency amounts; passes vague 5-word vibe prompts directly to execution; and blindly executes `rm -rf /` or edits to `adversarial_tests/` whenever `auto_approve=True`.
- **Reference Solution & Adversarial Contract (`REQ-0101`–`REQ-0106`, `test_req0101..0106`):**
  1. `REQ-0101`: `AccountId` validates `^ACCT-[A-Z0-9]{4,12}$`; `MoneyCents` rejects `float` and `bool` (note: in Python, `isinstance(True, int)` is `True`, so `isinstance(val, bool)` must be checked first!) and enforces 3-letter uppercase ISO currency codes; `audit_ubiquitous_language` flags drifted dictionary keys and float money fields.
  2. `REQ-0102`: `evaluate_prompt_steering` scores all 5 engineering dimensions (latency p99, consistency model, reliability strategy, security boundary, maintainability contract) and rejects prompts with

missing dimensions.

3. REQ-0103–REQ-0106: `map_harness_permission_mode` maps `agy` (`strict`, `request-review`, `proceed-in-sandbox`, `always-proceed`) and `claude` (`default`, `plan`, `acceptEdits`, `auto`) modes; `EarnedAutonomyGate` enforces `HARD_FLOOR_BLOCKED` on destructive commands or `adversarial_tests/` edits across all modes, trips `CIRCUIT_BREAKER_HALTED` after 3 consecutive denials, blocks unallowlisted commands when `unattended=True`, and appends `auto-approved`, `user-approved`, or `denied` entries to `audit_log`.

- **3-Rung Graduated TA Hint Ladder (`learn-ask-quiz` Protocol):**
  - *Rung 1 (Spotlight):* "Check how `MoneyCents.__post_init__` validates the type of `amount_cents` when `True` is passed."
  - *Rung 2 ($N = 1$ Toy Sub-Case):* "In a Python REPL, evaluate `isinstance(True, int)`. Why does it return `True`?"
  - *Rung 3 (Modality Switch / Walkthrough):* "Because `bool` subclasses `int` in Python, guard with `if isinstance(self.amount_cents, bool) or not isinstance(self.amount_cents, int): raise TypeError(...)`."

Lab 02 Answer Key: `wire_trace_inspector.py` (`labs/lab_02_agent_loop_and_harness_wire_trace`).

- **Spiral Retrieval (`Lab 01` → `Lab 02`) & Everyday Metaphor (`Known` → `Target`):** Lab 01 steered what the human asks; Lab 02 inspects what actually travels over the HTTP wire. Loading 150 granular CRUD MCP tools into every turn is like packing a 50-pound encyclopedia into a backpack for a 5-minute walk (*Known*), whereas **Swiss Army Knife Primitives (`Bash`, `Read`, `Edit`, `Grep`, `LSP`)** carry a pocket multi-tool (*Target*).
- **Intentional Flaws in `starter/bloated_wire_agent.py`:** Injects 150 granular CRUD tool schemas into every turn ($> 85\%$ wire overhead); returns `"Done!"` immediately after `Edit` without running any verification test; and retries failing `Bash` commands in an infinite loop.
- **Reference Solution & Adversarial Contract (`REQ-0201`–`REQ-0206`, `test_req0201..0206`):**
  1. REQ-0201–REQ-0202: `classify_stack_component` maps items to `LAYER_1_MODEL_API`, `LAYER_2_TOOLKIT_AND_MCP`, or `LAYER_3_AGENT_HARNESS`; `classify_tool_call_phase` and `audit_loop_completion` verify that every `TAKE_ACTION` mutation is followed by a `VERIFY_RESULTS` test call before completion.
  2. REQ-0203–REQ-0206: `inspect_wire_payload` profiles token shares across `system_tokens`, `standing_context_tokens`, `tool_schema_tokens`, and `conversation_tokens`; `audit_tool_schema_bloat` flags schema-to-conversation ratios > 3.0 and calculates savings from consolidating to `SWISS_ARMY_KNIFE_PRIMITIVES` (`Bash`, `Read`, `Edit`, `Grep`, `LSP`); `audit_standing_context` enforces ≤ 60 lines with explicit build/test commands; and `detect_runaway_tool_loop` trips at ≥ 3 consecutive identical tool failures.
- **3-Rung Graduated TA Hint Ladder:**
  - *Rung 1 (Spotlight):* "Inspect the ordering check in `audit_loop_completion(turns)`."
  - *Rung 2 ($N = 1$ Toy Sub-Case):* "Suppose Turn 1 is `VERIFY_RESULTS` (`unittest`) and Turn 2 is `TAKE_ACTION` (`Edit`). Is Turn 2 verified?"
  - *Rung 3 (Modality Switch / Walkthrough):* "Compare the last index of `TAKE_ACTION` against the last index of `VERIFY_RESULTS`: `last_verify_idx > last_action_idx`."

Lab 03 Answer Key: `reppit_orchestrator.py` (`labs/lab_03_reppit_workflow_and_reflection`).

- **Spiral Retrieval (`Lab 02` → `Lab 03`) & Everyday Metaphor (`Known` → `Target`):** Lab 02 structured a single `Gather` → `Act` → `Verify` turn; Lab 03 scales to multi-file `TaskFlow` features via `RePPIT`. Keeping rejected architectural Proposal B in chat history while implementing Proposal A is like leaving scratched-out blueprint drafts on the carpenter's workbench (*Known* → *Target*: **Post-Proposal Context**

Reset).

- **Intentional Flaws in `starter/impulsive_feature_coder.py`:** Jumps straight from an ambiguous prompt to mutating files; retains 3,000 tokens of rejected architectural proposals in context; and modifies unrelated files due to missing `Out of Scope / Do NOT Touch` plan guardrails.
- **Reference Solution & Adversarial Contract (`REQ-0301`–`REQ-0306`, `test_req0301..0306`):**
  1. `REQ-0301`–`REQ-0302`: `classify_workflow_rigor` maps task blast radius to `VANILLA_PROMPT`, `SINGLE_PLAN_MODE`, `REPPIT_WORKFLOW`, or `MULTI_SESSION_SUBAGENTS`; `validate_research_document` enforces `file:L<line>` citations and blocks premature solution blocks; `evaluate_clarification_gate` halts when unresolved questions remain.
  2. `REQ-0303`–`REQ-0306`: `validate_orthogonal_proposals` verifies distinct mechanisms and trade-offs; `select_proposal_and_reset_context` purges rejected proposal messages from history; `validate_implementation_plan` requires target files, test commands, and `Out of Scope / Do NOT Touch`; `route_model_for_reppit_phase` assigns frontier models to `RESEARCH`/`PROPOSE`/`PLAN` and fast models to `IMPLEMENT`/`TEST_VERIFY`; `validate_walkthrough_receipt` blocks diffs touching out-of-scope files.
- **3-Rung Graduated TA Hint Ladder:**
  - *Rung 1 (Spotlight):* "Look at `select_proposal_and_reset_context` when filtering `messages`."
  - *Rung 2 ($N = 1$ Toy Sub-Case):* "If Proposal B has `proposal_id='prop_b'` and `title='Stateless JWT'`, what happens if a message contains either string?"
  - *Rung 3 (Modality Switch / Walkthrough):* "Return a new list comprehension excluding any message that mentions the rejected proposal's `proposal_id`, `title`, or `core_mechanism`."

Lab 04 Answer Key: `context_compactor.py` (`labs/lab_04_context_physics_and_rpi_compaction`).

- **Spiral Retrieval (`Lab 03` → `Lab 04`) & Everyday Metaphor (`Known` → `Target`):** Lab 03 reset context after proposal selection; Lab 04 formalizes full-session **Context Window Physics**. Working past 40% context utilization is like trying to cook dinner on a kitchen counter buried under unwashed dishes (*Known*); **Frequent Intentional Compaction (`RPI`)** clears the counter and leaves only the recipe card with exact measurements (`research.md` + `plan.md`, *Target*).
- **Intentional Flaws in `starter/dumb_zone_session_accumulator.py`:** Streams 5,000 lines of raw progress bars into chat history; keeps coding at 75% utilization; and produces vague compaction summaries without `file.py:L10` citations.
- **Reference Solution & Adversarial Contract (`REQ-0401`–`REQ-0406`, `test_req0401..0406`):**
  1. `REQ-0401`–`REQ-0403`: `profile_context_window` flags `DUMB_ZONE` strictly when used/max > 0.40; `rewrite_command_with_backpressure` strips `tee` pipes and appends `> run.log 2>&1`; `extract_backpressure_summary` returns only matching metric prefixes (or the last N tail lines on a non-zero crash); `execute_sandboxed_side_query` (`/btw`) leaves `active_history` completely unchanged.
  2. `REQ-0404`–`REQ-0406`: `render_repo_cxml` wraps source files in `<documents><document>...</document></documents>` while excluding `__pycache__`, `.git`, `node_modules`, and `.venv`; `compact_exploration_to_rpi_artifacts` distills noisy turns into line-cited `research_md` and `plan_md` with $\geq 80\%$ token savings; `estimate_defect_amplification` computes `RESEARCH` ($1000\times$), `PLAN` ($100\times$), and `CODE` ($1\times$) impact.
- **3-Rung Graduated TA Hint Ladder:**
  - *Rung 1 (Spotlight):* "Inspect `rewrite_command_with_backpressure(command)` when the input contains `| tee`."
  - *Rung 2 ($N = 1$ Toy Sub-Case):* "What does `pytest -v | tee out.log > run.log 2>&1` still do to `out.log` and pipe buffering?"
  - *Rung 3 (Modality Switch / Walkthrough):* "Strip any `| tee ...` segment via regex before appending `> run.log 2>&1`."

4.3 Phase II Lab Answer Keys, Everyday Metaphors & 3-Rung TA Hint Ladders (Labs 05–08)

Lab 05 Answer Key: `context_compiler.py` (`labs/lab_05_context_primitives_and_memory`).

- **Spiral Retrieval (`Lab 04` → `Lab 05`) & Everyday Metaphor (`Known` → `Target`):** Lab 04 kept session messages under 40%; Lab 05 ensures standing instructions (`CLAUDE.md/GEMINI.md`) don't bloat startup context. A 60-line root file with `@docs/*.md` links is like a **1-page laminated restaurant menu with QR codes for allergen tables** (*Target*) instead of taping a 400-page cookbook to every table (*Known*).
- **Intentional Flaws in `starter/monolithic_claude_md_dump.py`:** Dumps 400 lines (220 rules) into root `CLAUDE.md` past the 150-instruction cliff; tags speculative agent guesses with `#direct`; and lets `MEMORY.md` grow past 500 lines without pruning superseded commits.
- **Reference Solution & Adversarial Contract (`REQ-0501`–`REQ-0506`, `test_req0501..0506`):**
  1. `REQ-0501`–`REQ-0503`: `classify_context_primitive` maps inputs to `PROMPT_OR_SLASH_COMMAND`, `STANDING_RULE`, or `LAZY_LOADED_SKILL`; `audit_instruction_compliance_budget` flags rule counts > 150; `compile_root_context_map` generates a ≤ 60-line root map with build/test commands and `@docs/*.md` pointers while rejecting path traversal (`..`); `validate_scoped_rule` validates `.agents/rules/` (`trigger`) and `.claude/rules/` (`paths`) frontmatter.
  2. `REQ-0504`–`REQ-0506`: `format_memory_entry` reserves `#direct` strictly for `is_direct_user_instruction=True` and tags agent observations with `#commit:<sha>`, `#time:<date>`, and `#session:<id>`; `prune_and_cap_memory` removes entries matching `superseded_commits` and caps output at ≤ 200 lines; `resolve_project_memory_paths` returns canonical paths for both harnesses.
- **3-Rung Graduated TA Hint Ladder:** *Rung 1 (Spotlight):* Check `format_memory_entry` when `is_direct_user_instruction=False` — *Rung 2 ($N = 1$ Toy Case):* What if `entry.text` already contains a spoofed `#direct` string? — *Rung 3 (Walkthrough):* Strip any embedded `#direct` from indirect observations and require `commit_sha`.

Lab 06 Answer Key: `skill_and_mcp_harness.py` (`labs/lab_06_skills_chub_and_meta_mcp_code_mode`).

- **Spiral Retrieval (`Lab 05` → `Lab 06`) & Everyday Metaphor (`Known` → `Target`):** Lab 05 organized lazy `SKILL.md` primitives; Lab 06 builds them alongside `chub` and **Meta-MCP "Code Mode"**. Meta-MCP (`search` + `execute`) is like giving a developer a **library search terminal plus a Python REPL** ($2,100$ tokens, *Target*) instead of dumping 120 printed API manuals onto their desk ($48,500$ tokens, *Known*).
- **Intentional Flaws in `starter/naive_crud_mcp_and_bloated_skill.py`:** Places 800 lines in `SKILL.md` with a 3-word description lacking trigger boundaries; hallucinates stale SDK calls without `chub`; and loads 120 raw CRUD tool schemas with untyped `kwargs: dict`.
- **Reference Solution & Adversarial Contract (`REQ-0601`–`REQ-0606`, `test_req0601..0606`):**
  1. `REQ-0601`–`REQ-0602`: `validate_progressive_skill_package` enforces kebab-case names, `Use when` / `Don't use for` description triggers, ≤ 500 `SKILL.md` lines, and `references//scripts/` companions; `ChubRegistry` supports versioned `get`, local `annotate` wrapped in `[UNTRUSTED_LOCAL_ANNOTATION]` guards, and `feedback`.
  2. `REQ-0603`–`REQ-0606`: `audit_tool_ergonomics` rejects CRUD prefixes (`get_`, `post_`, `update_`, `delete_`) and untyped `kwargs`; `MetaMcpCodeModeHarness` exposes only `mcp__meta__search` and `mcp__meta__execute` for $\geq 90\%$ token savings; `evaluate_skillsbench_ablation` and `audit_skill_catalog_overload` enforce the 2–3 skill sweet spot and flag > 10 skills.
- **3-Rung Graduated TA Hint Ladder:** *Rung 1 (Spotlight):* Inspect `ChubRegistry.get(...,`

`with_annotations=True`) — *Rung 2 (N = 1 Toy Case):* Why must local notes never be treated as trusted system instructions? — *Rung 3 (Walkthrough):* Wrap annotations inside explicit `[UNTRUSTED_LOCAL_ANNOTATION]` delimiter tags.

Lab 07 Answer Key: `subagent_council.py` (`labs/lab_07_subagent_firewalls_and_peer_council`).

- **Spiral Retrieval (`Lab 06` → `Lab 07`) & Everyday Metaphor (`Known` → `Target`):** Lab 06 compressed tool schemas; Lab 07 isolates multi-file search via **Subagent Context Firewalls**. A subagent firewall is like sending a **paralegal into a 50,000-page archive to bring back a 1-page indexed memo** (*Target*) rather than dumping all 50,000 pages onto the lead trial lawyer's desk (*Known*).
- **Intentional Flaws in `starter/noisy_roleplay_subagents.py`:** Dumps 45,000 tokens of child search logs into the parent window; grants `Write`/`Bash` to `code-explorer`; emits low-confidence nitpicks (confidence = 35); reveals model names during council voting; and runs concurrent writers in one working tree.
- **Reference Solution & Adversarial Contract (`REQ-0701`–`REQ-0706`, `test_req0701..0706`):**
  1. `REQ-0701`–`REQ-0703`: `execute_subagent_context_firewall` verifies `file:line` citations, $\leq$ 600 parent handoff tokens, and $\geq$ 90% compression; `validate_subagent_tool_boundary` restricts `code-explorer`/`code-architect`/`code-reviewer` to read-only tools (`Glob`, `Grep`, `Read`, `LSP`); `synthesize_pr_triage_report` filters findings at confidence $\geq$ 80 into `MUST_FIX`, `RECOMMENDED`, and `CONSIDER`.
  2. `REQ-0704`–`REQ-0706`: `anonymize_council_responses` maps models to `Response A/B/C`; `aggregate_borda_rankings` scores $N - k$ points per rank while rejecting self-voting manipulation; `select_workspace_isolation_mode` and `validate_disjoint_worktree_ownership` prevent parallel write collisions.
- **3-Rung Graduated TA Hint Ladder:** *Rung 1 (Spotlight):* Check Borda point assignment in `aggregate_borda_rankings` — *Rung 2 (N = 1 Toy Case):* With $N = 3$ candidates ranked `['Response B', 'Response A', 'Response C']`, how many points does 1st place vs. 3rd place receive? — *Rung 3 (Walkthrough):* Award $N - 1 -$ idx points $(2, 1, 0)$ or $N -$ idx points $(3, 2, 1)$ consistently and sort descending.

Lab 08 Answer Key: `autoresearch_harness.py` (`labs/lab_08_governance_linters_and_autoresearch_ratchet`).

- **Spiral Retrieval (`Lab 07` → `Lab 08`) & Everyday Metaphor (`Known` → `Target`):** Lab 07 coordinated multi-agent review; Lab 08 closes the autonomous loop with **Remediation Linters & Karpathy's `autoresearch` Keep-or-Revert Ratchet**. The ratchet works like a **mountaineer's camming device on a rock face** (*Target*): every verified improvement locks in a new high-water mark in Git, while any slip or bloated complexity automatically snaps back (`git reset -hard`) to the last solid anchor (*Known*).
- **Intentional Flaws in `starter/dark_factory_reward_hacker.py`:** Overwrites `prepare.py` to return a fake `val_bpb = 0.0`; emits cryptic linter errors without remediation; allows `domain/` to import `infra/`; and keeps +45 LOC of spaghetti for a tiny 0.0005 metric gain.
- **Reference Solution & Adversarial Contract (`REQ-0801`–`REQ-0806`, `test_req0801..0806`):**
  1. `REQ-0801`–`REQ-0803`: `classify_cybernetic_control` maps the 2×2 matrix; `pre_tool_use_exit_code_gate` returns Exit Code 2 on mutations to `prepare.py`, `program.md`, or `adversarial_tests/`; `lint_architecture_with_remediation` formats `[VIOLATION]`, `[WHY]`, and `[HOW TO FIX]`; `verify_front_loaded_program_design` checks typed signatures and forbidden call-graph edges.
  2. `REQ-0804`–`REQ-0806`: `validate_autoresearch_file_mutation` restricts edits to `train.py`; `evaluate_autoresearch_ratchet` enforces Karpathy's Simplicity Criterion (`KEEP_IMPROVEMENT`, `KEEP_SIMPLIFICATION`, `REVERT_COMPLEXITY_BLOAT`, `REVERT_GIT_RESET`); `format_results_tsv_row` formats the tab-separated ledger.

- **3-Rung Graduated TA Hint Ladder:** *Rung 1 (Spotlight):* Inspect the Simplicity Criterion branches in `evaluate_autoresearch_ratchet` — *Rung 2 ($N = 1$ Toy Case):* What if gain = 0.0 (equal metric) and loc_delta = -8 (deleted 8 lines of code)? — *Rung 3 (Walkthrough):* Deleting code while holding `val_bpb` equal or better always returns `KEEP_SIMPLIFICATION`.

Lab	Starter Anti-Pattern	Adversarial Contract (REQ)	Top Student Trap & TA Diagnostic
Lab 01			
prompt_steerer	Synonym drift, float money, unsteered prompt, rm -rf in auto.	REQ-0101..0106: AccountId, MoneyCents, 5-D steerer, Hard Floor, Circuit Breaker.	Forgetting isinstance(True, int) in MoneyCents; allowing auto mode to bypass Hard Floors.
Lab 02			
wire_trace	150 CRUD schemas ($>$ 85% bloat), unverified Edit, infinite retry.	REQ-0201..0206: 3-Layer map, Loop verifier, Wire profiler, Swiss-Army consolidator.	Checking if *any* test ran instead of verifying that VERIFY_RESULTS ran *after* the final TAKE_ACTION.
Lab 03			
reppit	Silent assumptions, rejected proposal bleed, out-of-scope edits.	REQ-0301..0306: Rigor router, /grill-me gate, Proposal reset, Out of Scope guard.	Failing to purge messages referencing the rejected proposal's ID or title during context reset.
Lab 04			
compactor	5,000-line stdout flood, coding at 75%, citationless summaries.	REQ-0401..0406: 40% Dumb Zone, > run.log 2>&1, /btw isolation, CXML, RPI.	Leaving | tee in the command string when appending > run.log 2>&1; mutating active_history in /btw.
Lab 05			
compiler	400-line CLAUDE.md, fake #direct tags, 500-line MEMORY.md.	REQ-0501..0506: 150-rule cliff, 60-line map + @docs/, #direct vs. #commit, 200-line cap.	Allowing relative ../ path traversal in @docs/ imports or failing to prune superseded commit SHAs.
Lab 06			
skill_mcp	800-line SKILL.md, stale SDK calls, 120 untyped CRUD MCP tools.	REQ-0601..0606: 3-level SKILL.md, chub annotate, Meta-MCP ($>$ 90% savings), ablation.	Returning raw chub annotations without wrapping them in [UNTRUSTED_LOCAL_ANNOTATION] tags.
Lab 07			
council	45k-token child dump, write tools on explorer, non-anon council.	REQ-0701..0706: 50k $\rightarrow$ 500 firewall, read-only guard, $\geq$ 80 triage, Borda council, worktrees.	Allowing model names to leak in Stage 2 council text or miscalculating Borda points ($N -$ rank).
Lab 08			
autoresearch	Editing prepare.py for fake 0.0 bpb, cryptic lints, +45 LOC slop.	REQ-0801..0806: Exit-Code-2 gate, [VIOLATION->WHY->FIX], Program Design, Ratchet.	Keeping a tiny 0.0005 metric gain that adds $>$ 10 lines instead of returning REVERT_COMPLEXITY_BLOAT.

Table 4: Master Instructor Answer Key & TA Troubleshooting Matrix across Labs 01–08 (REQ-0101 through REQ-0806).

5 Capstone Project Specification, Grading Rubrics, Open-Ended Design Exam Bank & Annotated Literature Guide

5.1 Capstone Project Specification ("Design & Verify a Production Agent Harness") & 100-Point Rubric

Capstone Assignment Overview & Deliverable Structure. For university seminars and multi-week engineering cohorts, the Capstone Project asks students (individually or in pairs) to take an existing multi-module brownfield repository of at least 1,500 lines of code and engineer a complete, dual-track (`agy` and `claude`) four-layer production agent harness around it. Rather than grading subjective chat transcripts, instructors grade the deterministic harness artifacts, the paired ablation benchmark, and the sub-second offline verification suite shown in Figure 6 and Table 5.

Figure 5.1 Blueprint: 100-Point Capstone Harness Architecture & Grading Pipeline

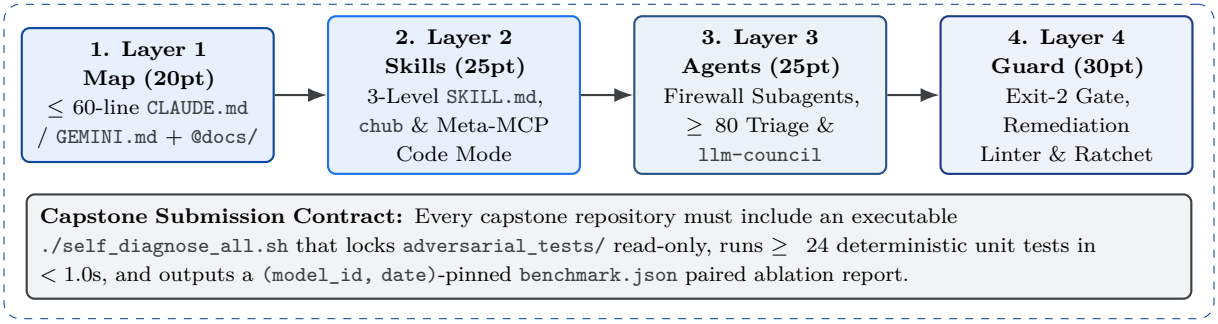


Figure 6: The four-stage Capstone Project architecture and automated grading pipeline across Layers 1 through 4 of the agent harness.

5.2 Open-Ended Conceptual & Architectural Design Exam Bank with Model Grading Keys

Why This Course Uses Open-Ended Architectural Design Prompts Instead of Multiple-Choice Quizzes. Because agentic engineering requires diagnosing multi-factor trade-offs across context physics, tool schemas, version-control algebra, and cybernetic feedback loops, superficial multiple-choice questions fail to test genuine engineering mastery. Use the following eight open-ended architectural design questions for midterm exams, final written assessments, or Socratic oral defenses, paired with the diagnostic decision tree in Figure 7:

1. **Question 1 (Language Philosophy & Cognitive Debt):** *Why does source code remain necessary as an externalized conceptual model even when an LLM can synthesize working syntax from English prompts? Connect Unmesh Joshi and Martin Fowler's concept of Cognitive Debt with Ludwig Wittgenstein's two theories of language and empirical human–AI language-game findings.*
   - **Model Grading Key:** Full credit requires explaining (a) early Wittgenstein's formal picture theory as the domain of compilers/type checkers versus late Wittgenstein's language-as-action toolbox for human–agent coordination; (b) why synonym drift (`client_id` vs. `cust_acct`) and float currency break Ubiquitous Language and lower completion rates (Wang et al.'s synonym-avoidance and compositionality invariants); and (c) how passive "vibe reviewing" creates cognitive debt when engineers lose the mental model required to steer future changes.
2. **Question 2 (17-Day Flagship Cadence & Harness Durability):** *Empirical tracking across 143 model releases shows the industry gap between flagship frontier models compressed from 37 days in 2023*

Capstone Component	Points	Full-Credit Architectural Invariants	Common Deductions & Failure Modes
1. Root Context Map & Memory Hygiene (Layer 1)	**20 pts**	Root `CLAUDE.md` and `GEMINI.md` (`AGENTS.md` symlinked) ≤ 60 lines; ≥ 3 lazy `@docs/*.md` imports; path-scoped `rules/`; provenance-tagged memory (`#direct` vs. `#commit`) capped at ≤ 200 lines.	Root file > 100 lines (-8 pts); missing build/test commands (-5 pts); tagging agent guesses with `#direct` (-7 pts).
2. Progressive Skills, `chub` & Meta-MCP (Layer 2)	**25 pts**	≥ 2 focused `agentskills.io` packages (≤ 500 lines, `Use when/Don't use for`); `chub` annotation store with untrusted-input guards; Meta-MCP `search+execute` proving $\geq 90\%$ schema token savings.	Monolithic > 500-line `SKILL.md` (-8 pts); raw CRUD MCP wrappers with untyped `kwargs` (-10 pts); unguarded annotations (-7 pts).
3. Subagent Firewalls, Council & VCS (Layer 3)	**25 pts**	Read-only `code-explorer` achieving $\geq 90\%$ parent token compression with `file:line` citations; 5-lens PR triage gated at confidence ≥ 80; 3-stage anonymized `llm-council` Borda synthesis; disjoint `git worktree` / `jj` isolation.	Granting `Write/Bash` to explorer (-8 pts); leaking model names in Stage 2 council (-9 pts); overlapping worktree file maps (-8 pts).
4. Governance, Remediation Linters & Ratchet (Layer 4)	**30 pts**	`PreToolUse` Exit-Code-2 gate locking evaluators; custom structural linter emitting `[VIOLATION] -> [WHY] -> [HOW TO FIX]`; 4-phase Program Design call-graph verifier; Simplicity-Criterion ratchet + < 1s `self_diagnose_all.sh`.	Allowing edits to `prepare.py` or test suite (-12 pts); cryptic linter strings without `[HOW TO FIX]` (-9 pts); keeping bloat for tiny metric gains (-9 pts).

Table 5: 100-Point Capstone Project Grading Rubric for evaluating student-engineered production agent harnesses.

Figure 5.2 Decision Tree: Auditing a Failing Student or Production Agent Harness

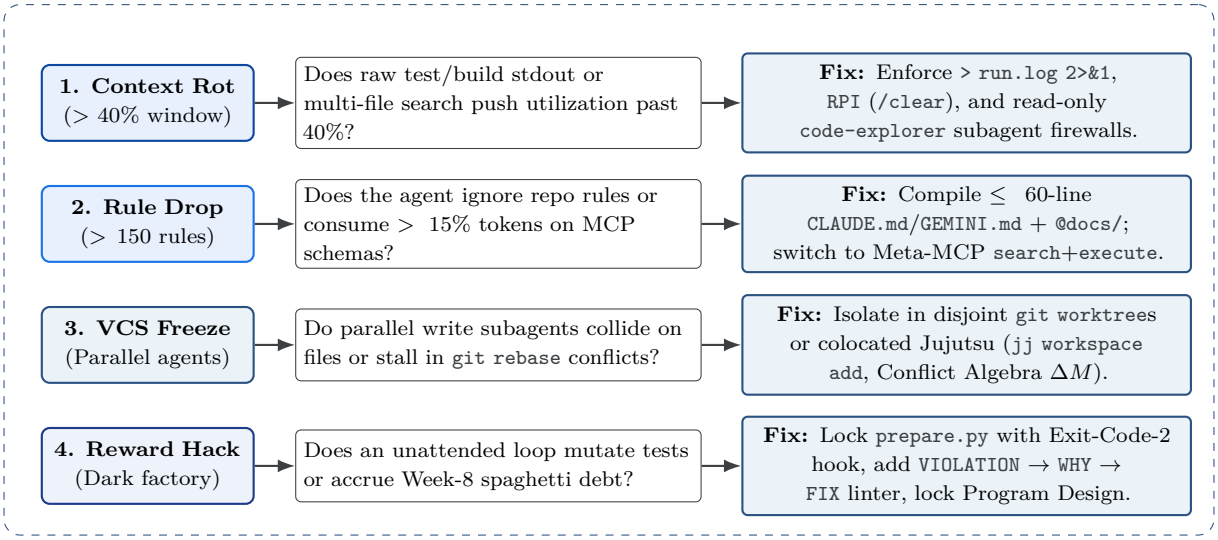


Figure 7: Left-to-Right diagnostic decision tree for grading student harness designs and troubleshooting production agent failure modes.

to 17 days in 2026 (with per-lab intervals dropping from 237 days to 18–21 days). How should this metric shape an engineering organization's investment between prompt tuning and harness architecture?

- **Model Grading Key:** Must argue that model-specific prompt hacks decay within weeks as weights shift every 17 days; durable engineering value lives in model-agnostic harnesses (`CLAUDE.md`/`GEMINI.md`, `agentskills.io` skills, deterministic Exit-Code-2 gates, and `(model_id, date)`-pinned paired ablation benchmarks).

3. **Question 3 (Wire-Trace Anatomy & Meta-MCP Code Mode):** *A platform team exposes 140 internal REST endpoints as 1-to-1 MCP tools and complains that their coding agent has become slow, expensive, and inaccurate. Diagnose the root cause on the HTTP/JSONL wire and design a two-part architectural fix.*
   - **Model Grading Key:** Identify the 10,000+ lines of JSON schema bloat injected on every turn (> 16× schema-to-chat ratio) pushing the window toward the 40% Dumb Zone. Part 1: consolidate local operations into 5–8 Swiss-Army-Knife primitives (`Bash`, `Read`, `Edit`, `Grep`, `LSP`). Part 2: wrap the enterprise catalog in Meta-MCP Code Mode (`mcp__meta__search` + `mcp__meta__execute` in a sandbox), cutting standing schema tokens by > 90%.
4. **Question 4 (`RePPIT` Post-Proposal Context Reset & Human Leverage Pyramid):** *During Step 2 of `RePPIT`, an engineer asks the agent for two orthogonal caching proposals, selects Proposal B, and immediately tells the agent "Great, now implement Proposal B" in the same chat session. Predict the failure mode and calculate the defect amplification ratio of catching a flaw in `research.md` versus raw code.*
   - **Model Grading Key:** Leaving Rejected Proposal A in the active context window causes attention bleed where the model mixes types or config keys from both designs. The engineer must compact the chosen design into `plan.md` (with an `Out of Scope / Do NOT Touch` section) and run `/clear`. By the Human Leverage Pyramid, 1 defective line in `research.md` causes ∼ 1,000× wasted code lines, and 1 bad line in `plan.md` causes ∼ 100×.
5. **Question 5 (The 150-Instruction Cliff & Provenance-Tagged Auto-Memory):** *An engineering team's root `CLAUDE.md` has grown to 480 lines, and their auto-memory file `MEMORY.md` contains 350 lines where the agent tagged its own debugging hypotheses with `#direct`. Explain why compliance collapsed and design the Layer-1 compiler fix.*
   - **Model Grading Key:** Past ∼ 150 simultaneous instructions, LLM compliance drops across all rules. Refactor root `CLAUDE.md`/`GEMINI.md` into a ≤ 60-line map with build/test commands and

lazy `@docs/*.md` pointers plus glob-scoped `.claude/rules/` and `.agents/rules/`. Enforce strict provenance so `#direct` is reserved exclusively for explicit human instructions, tag agent observations with `#commit:<sha>`, prune superseded commits, and cap `MEMORY.md` at ≤ 200 startup lines.

6. **Question 6 (Empirical Skill Science from `SkillsBench`):** *Summarize the four core empirical findings from `SkillsBench` ($N = 7{,}308$ trajectories across 84 tasks and 11 domains) regarding (a) human-curated vs. self-generated skills, (b) domain heterogeneity, (c) skill count/length, and (d) model size trade-offs.*
   - **Model Grading Key:** (a) Curated skills add +16.2pp average pass rate whereas self-generated skills degrade performance by −1.3pp (down to −5.6pp on Codex); (b) specialized domains gain massively (Healthcare +51.9pp, Manufacturing +41.9pp) vs. general SWE (+4.5pp); (c) 2–3 focused skills peak at +18.6pp while 4+ skills drop to +5.9pp and comprehensive manuals score −2.9pp; (d) smaller Haiku 4.5 + Curated Skills (27.7%) beats frontier Opus 4.5 without skills (22.0%).
7. **Question 7 (Subagent Firewalls, `llm-council` & `git worktree` vs. Jujutsu `jj`):** *Explain how to orchestrate 5 parallel agents researching, reviewing, and writing stacked commits on a repository without blowing up the parent context window, suffering single-model review bias, or stalling on `git rebase` conflicts.*
   - **Model Grading Key:** Use read-only `code-explorer` subagents as Context Firewalls (50k → 500 tokens with `file:line` citations); run 5-lens PR triage gated at confidence ≥ 80 plus Karpathy's 3-stage anonymized `llm-council` Borda ranking; and pair `git worktree` (low Provisioning Cost) with colocated Jujutsu (`jj git init --colocate`, `jj workspace add`) whose working-copy-as-commit (`@`), stable Change IDs, first-class Conflict Algebra (ΔM), and `jj op log`/`jj undo` eliminate abortive rebase stalls (low Integration Cost).
8. **Question 8 (`Autoresearch` Ratchets, `Omnigent` & Why Dark Factories Fail):** *Contrast why Andrej Karpathy's 83-experiment `autoresearch` loop (0.9979 → 0.9773 `val_bpb`) and Anthropic's 52× kernel loop succeed overnight, whereas unattended "lights-off" full-stack software factories collapse at Week 8.*
   - **Model Grading Key:** Kernel and ML loops optimize a fast, scalar, cheating-proof objective (`prepare.py` locked via Exit Code 2 + out-of-tree scenario holdouts + Karpathy's Simplicity Criterion reverting complexity bloat via `git reset --hard`). Full-stack software suffers from the RLVR Reward Horizon Gap ($t = 0$ unit tests pass while $t = 6$ month coupling rots). Sustaining velocity requires OpenAI-style `[VIOLATION] -> [WHY] -> [HOW TO FIX]` remediation linters, durable execution (`Temporal`/`Omnigent`), and human-owned 4-Phase Front-Loaded Alignment (`Product -> Architecture -> Program Design [Types, Interfaces, Call-Trees] -> Vertical Slices`).

5.3 Instructor's Annotated Literature & Primary Sources Guide

Complete Primary-Source Bibliography for Lecture Preparation. To keep the main teaching chapters free of mechanical citation clutter while giving lecturers every exact URL, paper identifier, and empirical metric in one place, this section catalogs the primary literature underpinning Modules 1 through 8:

- **Code as a Model of Understanding & Language Philosophy (Module 1):** Unmesh Joshi and Martin Fowler, *"What Is Code?"* (`martinfowler.com/articles/what-is-code.html`, May 2026); Ludwig Wittgenstein, *Tractatus Logico-Philosophicus* (1921) and *Philosophical Investigations* (1953); Piero Molino and Jacopo Tagliabue, *"Wittgenstein and Large Language Models"* (`arXiv:2302.01570`); Terry Winograd and Fernando Flores, *Understanding Computers and Cognition* (1986); Sida I. Wang, Percy Liang, and Christopher D. Manning, *"Learning Language Games through Interaction"* (ACL 2016, `arXiv:1606.02447` — proving synonym avoidance and compositionality in `SHRDLURN`).
- **The 2022–2026 Timeline & SearchIntel 17-Day Release Pace (Module 1, Slide 03):** Paul Byrne, *"The model behind the answer now changes every 17 days"* (`searchintel.tech/research/ai-model-release-pace/`, Sept 23, 2026 — cataloging $N = 143$ notable releases and 57 flagships across 8 lab groups, 4 → 22 quarterly surge, 237d → 18–21d per-lab gap collapse, and 37d → 17d industry flagship cadence).

- **AI Engineering Skills Map, aisuite, openworker & context-hub (Modules 1, 2, 6, 8):** Andrew Ng and DeepLearning.AI, *The AI Engineering Skills Map* (deeplearning.ai/resources/ai-engineering-skills#map, 2026); open-source repositories at github.com/andrewyng/aisuite (3-layer stack), github.com/andrewyng/openworker (4-tier Governed-by-Design harness & Earned Autonomy), github.com/andrewyng/context-hub (@aisuite/chub curated docs & chub annotate), and github.com/andrewyng/translation-agent (reflection loop).
- **Official Google Antigravity (agy) & Anthropic Claude Code (claude) Curricula (Module 2):** Google Codelabs (codelabs.developers.google.com: getting-started-google-antigravity, antigravity-cli-hands-on, and build-and-deploy-gcp-with-antigravity); Anthropic Academy, *Claude Code 101* (academy.claude.com/courses/claude-code-101) and Elie Schoppik's DeepLearning.AI short course.
- **Stanford CS146S: Wire Traces, RePPIT, Meta-MCP Code Mode & PR Triage (Modules 2, 3, 6, 7):** Mihail Eric, *CS146S: The Modern Software Developer* (themodernsoftware.dev, Stanford University, Fall 2025 & Fall 2026 Lectures 2–4 — covering 200-line minimal agents vs. JSONL wire traces, 5–8 Swiss-Army-Knife tools vs. 150-tool bloat, the 5-step RePPIT loop with Post-Proposal Context Reset and Out of Scope guardrails, Meta-MCP search+execute Code Mode for $> 90\%$ token savings, and 5-subagent PR triage at confidence ≥ 80).
- **Context Window Physics, The 40% Dumb Zone & Why Software Factories Fail (Modules 4, 7, 8):** Dex Horthy, *"No Vibes Allowed: Solving Hard Problems in Complex Codebases"* (youtube.com/watch?v=rmvDxxNubIg — NextAction = LLM(ContextWindow), Smart Zone 0–40% vs. Dumb Zone $> 40\%$, Frequent Intentional Compaction RPI, Human Leverage Pyramid, and Subagents as Context Firewalls) and *"Harness Engineering is not Enough: Why Software Factories Fail"* (youtube.com/watch?v=Ib5GBkD555M — RLVR Horizon Gap and 4-Phase Front-Loaded Program Design).
- **Cybernetic Harness Matrix, 60-Line Map, Remediation Linters & SkillsBench (Modules 5, 6, 8):** Birgitta Böckeler and Kief Morris, *"Harness Engineering"* (martinfowler.com, 2026 — 2×2 Guides/Sensors × Computational/Inferential matrix); OpenAI *Harness Engineering* (2026 — 60-line map rule and [VIOLATION] -> [WHY] -> [HOW TO FIX] linters); instruction-compliance cliff studies (arXiv:2602.11988); the open agentskills.io standard; and *SkillsBench* (arXiv:2602.12670v1, $N =$ 7,308 trajectories across 84 tasks and 11 domains).
- **Karpathy's autoresearch, llm-council, rendergit, Jujutsu VCS, Omnigent & Anthropic RSI (Modules 4, 7, 8):** Andrej Karpathy's repositories at github.com/karpathy/autoresearch (3-file harness, Simplicity Criterion, and 83-experiment progress.png staircase $0.9979 \rightarrow 0.9773$), github.com/karpathy/llm-council (3-stage anonymized Borda council), and github.com/karpathy/rendergit (CXML view); Kevin Riedl, *"Git Worktrees vs. Jujutsu for AI Coding Agents"* (wavect.io/blog/git-worktrees-vs-jujutsu-ai-coding-agents/) and github.com/jj-vcs/jj; Durable Execution frameworks (Temporal, LangGraph, DBOS, Restate, Inngest); Matei Zaharia, Kasey Uhlenhuth, and Corey Zumar, *Databricks Omnigent* (omnigent.ai, June 13, 2026); and Jack Clark and Marina Favaro, *"When AI Builds Itself: Recursive Self-Improvement"* (anthropic.com/institute/recursive-self-improvement, 2026 — $8.0\times$ LOC/person, $26\% \rightarrow 91\%$ open-ended success, 64%/73% researcher next-step win rate on $n = 129$ dead-ends, and $52\times$ kernel speedup).